

HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE

HENRY DAVID KEE

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY

JANUARY 2026

HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE

BY

HENRY DAVID KEE

THE PROJECT REPORT IS PREPARED FOR
FACULTY OF INFORMATION SCIENCE & TECHNOLOGY
MULTIMEDIA UNIVERSITY
IN PARTIAL FULFILLMENT
FOR

BACHELOR OF INFORMATION TECHNOLOGY
(HONS) SECURITY TECHNOLOGY

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY

JANUARY 2026

© 2026 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED

Copyright of this report belongs to Universiti Telekom Sdn. Bhd as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialization policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

DECLARATION

I hereby declare that the work have been done by myself and no portion of the work contained in this thesis has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.



HENRY DAVID KEE

Faculty of Information Science & Technology
Multimedia University

Submission Date: 22: 01: 2026

Submission Time: 12: 00

ACKNOWLEDGEMENT

I would like to thank everyone who had contributed to the successful completion of this project. I would like to express my gratitude to my supervisor, Ts. Ibrahim Yusof for his invaluable advice, guidance and his enormous patience throughout the development of the research.

In addition, I would also like to express my gratitude to my loving parent and friends who had helped and given me unwavering support during this challenging journey.

ABSTRACT

In an age of escalating cyber threats, organizations require proactive methods to detect, understand, and counteract evolving attack patterns. Honeypots serve as decoy systems designed to lure attackers, enabling researchers and security teams to gather valuable threat intelligence in a controlled environment. This project aimed to deploy and analyze a multi-honeypot platform using the T-Pot framework, which integrates several honeypots such as Cowrie and Dionaea, to simulate services like SSH, SMB, and HTTP. The system was complemented by the ELK Stack (Elasticsearch, Logstash, Kibana) to collect, process, and visualize attacker logs in real time, allowing for comprehensive insight into attacker behavior and tactics.

The methodology follows the Waterfall model, ensuring a structured progression through phases including requirements gathering, system analysis, design, implementation, testing, deployment, and maintenance. Detailed system architecture diagrams and deployment workflows were designed to guide the technical execution. Wireframe prototypes were also developed to illustrate user interactions with the Kibana-based dashboard.

Real-world simulations were conducted to validate honeypot performance. Attack patterns were analyzed using both log inspection and MITRE ATT&CK mapping. Data captured revealed key insights into brute-force attempts, malware drop patterns, and attacker geolocations. Several case studies were referenced to benchmark the deployment against existing systems and evaluate the effectiveness of multi-level interaction honeypots.

In conclusion, the T-Pot honeypot environment that was successfully deployed was effectively gathered actionable threat intelligence and demonstrate the advantages of using containerized, integrated honeypots for real-time security monitoring. The system analyzed 329 events, identifying a major botnet campaign originating from South Korea. This research contributes toward practical implementation guidance and further underscores the value of honeypots in modern cybersecurity defense strategies.

TABLE OF CONTENTS

DECLARATION	III
ACKNOWLEDGEMENT	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF TABLES.....	IX
LIST OF FIGURES.....	X
LIST OF ABBREVIATIONS/ SYMBOLS	XII
LIST OF APPENDICES.....	XIII
CHAPTER 1 INTRODUCTION.....	1
1.1 Overview	1
1.2 Problem Statement	2
1.3 Project Objectives	2
1.4 Project Scope.....	2
1.5 Report Organisation	3
1.6 Gantt Chart for FYP 1	4
1.7 Gantt Chart for FYP 2	5
CHAPTER 2 LITERATURE REVIEW	6
2.1 Overview	6
2.2 Honeypot Types and Tools	6
2.2.1 Honeypot Classification	6
2.2.2 Comparison of Honeypot Tools	10
2.3 Threat Intelligence Methodologies.....	12
2.4 Case Studies of Honeypot implementation	17
CHAPTER 3 METHODOLOGY	23
3.1 Overview	23
3.2 Waterfall model.....	23

3.3	Software/tools Required	29
3.4	Hardware Required	33
CHAPTER 4 SYSTEM DESIGN.....		34
4.1	Overview	34
4.2	System Architecture Diagram	35
4.3	Deployment Workflow Diagram.....	36
4.4	Data Flow Diagram	38
4.5	Use Case Diagram	40
4.6	Wireframe Design	41
4.6.1	Dashboard Overview	41
4.6.2	Log Drilldown View	42
CHAPTER 5 IMPLEMENTATION		44
5.1	Overview	44
5.2	Cloud Environment and Infrastructure Setup.....	45
5.3	Deployment of T-Pot Framework	49
5.4	Development of Threat Intelligence Enrichment Engine.....	52
5.5	MITRE ATT&CK Mapping Module	54
5.6	Threat Feed Generation and MISP Integration	57
5.7	System Verification and Testing.....	61
CHAPTER 6 RESULTS AND DISCUSSION		63
6.1	Data Collection Overview	63
6.2	Data Collection Analysis.....	63
6.3	Threat Intelligence Enrichment.....	65
6.4	MITRE ATT&CK Classification	66
6.5	Case Study: The "Korean Botnet" Analysis.....	68
6.6	Real-World Security Application.....	69
CHAPTER 7 CONCLUSION		70
7.1	Conclusion.....	70
7.2	Limitations	71
7.3	Future Work	71

REFERENCES	73
APPENDICES.....	74

LIST OF TABLES

Table 1.1: Gantt Chart FYP 1	4
Table 1.2: Gantt Chart FYP 2	5
Table 2.1: Comparison between interaction levels.....	9
Table 2.2: Contrast between case studies	22
Table 6.1: Distribution of Captured Events by Service	63

LIST OF FIGURES

Figure 2.1: Low-Interaction Honeypot.....	7
Figure 2.2: Medium-Interaction Honeypot.....	8
Figure 2.3: High-Interaction Honeypot.....	9
Figure 2.4: T-Pot Honeypot.....	11
Figure 2.5: ELK Stack	12
Figure 2.6: MITRE ATT&CK Framework Matrix	14
Figure 3.1: The phases of the waterfall model	23
Figure 3.2: Docker and Docker Compose	29
Figure 3.3: ELK Stack (ElasticSearch, LogStash and Kibana)	30
Figure 3.4: Wireshark Interface	30
Figure 3.5: MITRE ATT&CK Navigator	31
Figure 3.6: Ubuntu	31
Figure 3.7: Oracle Virtual	32
Figure 4.1: System Architecture Diagram	35
Figure 4.2: Deployment Workflow Diagram	36
Figure 4.3: Data Flow Diagram.....	38
Figure 4.4: Use Case Diagram.....	40
Figure 4.5: Wireframe – Dashboard Home Overview	41
Figure 4.6: Wireframe – Log Drilldown View	42
Figure 5.1: DigitalOcean Setup	45
Figure 5.2: DigitalOcean CPU Setup	46
Figure 5.3: DigitalOcean Machine Created	47
Figure 5.4: Firewall Configurations	48
Figure 5.5: Docker pulling Honeypots.....	49
Figure 5.6: Kibana Dashboard.....	50
Figure 5.7: Elastic Event Logs.....	51
Figure 5.8: Log Extraction and Parsing Script.....	53
Figure 5.9: AbuseIPDB Integration.....	53
Figure 5.10: Mitre Script Execution	54
Figure 5.11: Mitre Script Results.....	55
Figure 5.12: MISP Connector Script Execution	57

Figure 5.13: MISP Script Output File	57
Figure 5.14: MISP Script Output File Data Structure	58
Figure 5.15: MISP Integration Script Update	59
Figure 5.16: MISP Integration Script Execution.....	60
Figure 5.17: MISP Integration Script Results	60
Figure 6.1: Distribution of Events by Honeypot Service	64
Figure 6.2: Top 5 Attacking Countries by Event Volume	65
Figure 6.3: Distribution of MITRE ATT&CK Techniques.....	66

LIST OF ABBREVIATIONS/ SYMBOLS

FIST	Faculty of Information Science and Technology
FYP	Final Year Project
MMU	Multimedia University
T-Pot	Threat Pot (Multi-honeypot framework)
ELK	Elasticsearch, Logstash, Kibana
VM	Virtual Machine

LIST OF APPENDICES

Appendix A: FYP Meeting Log.....	74
Appendix B: Checklist for FYP2 Final Submission	80
Appendix C: Technical Paper	82
Appendix D: Script Source Code	85

CHAPTER 1

INTRODUCTION

1.1 Overview

In today's world, honeypots have been playing a crucial role in collecting and retrieving newest cyberattacks and digital crime activities, and it has only grown even larger as the increase in accessibility to the internet. Causing many companies, banks, government agencies to get the attention of cyberthreats and crimes occurring right now by utilizing the use of honeypots that act as decoys, which are vulnerable to attract hackers to exploit them. While these deployed honeypots do not contain any valuable assets of the organization, hackers are exposed with their information details from their exploitation methods to their IP addresses. Allowing honeypots to gain real-time updates on malicious activities and even other threat information details that benefits other cybersecurity tools to prevent most of the possible cyberattacks.

This project focused on deploying a multi-honeypot environment using the T-Pot framework, integrating tools like Cowrie and Dionaea honeypot. The system simulated services such as SSH, HTTP, and SMB to attract attackers, while the ELK Stack (Elasticsearch, Logstash, Kibana) processes and visualizes attack data in real time. The goal was to generate actionable threat intelligence that can inform firewall rules, blocklists, and incident response strategies.

1.2 Problem Statement

Traditional intrusion detection systems (IDS) are often not able to detect newly discovered or unknown previous attack vectors and threat methods that are able to exploit newer computer systems and technologies' vulnerabilities. As numerous new computer systems and technologies are being developed, the increase of new cyber attacks and threat actors are drastically evolving in a rapid pace. With these issues, many companies and organizations begin to develop honeypots to address this gap by mimicking vulnerable services, which attracts threat actors to be tempted to try to exploit the honeypots while capturing the activities and behaviours of novel threat actors and unknown threat actors. But existing solutions lack integration with modern threat intelligence platforms and real-time analytics.

1.3 Project Objectives

The objectives of this project are as shown below:

- To set up a honeypot system to attract and log malicious activity
- To analyse the collected data to identify common attack patterns and trends
- To use the findings to improve defensive measures and share insights with the cybersecurity community.

1.4 Project Scope

This project was to develop and deploy a honeypot with threat intelligence. There were additional features after deploying this honeypot, which is a dedicated dashboard for the user to get an overall information updates of the latest activity recorded. This allowed the user to be able to get a heads-up on real-time malicious activities occurring to the honeypot deployed. Besides, by utilizing open-source tools like T-pot and ELK Stack to ensure cost-effectiveness.

1.5 Report Organisation

Chapter1:

Introduce the project overview, problem statement, objective, and project scope.

Chapter2:

Literature review on the different types of honeypots deployments, threat intelligence frameworks and case studies.

Chapter3:

The methodology that describes how this project will be implemented, as well as the tools and deployment steps to be done.

Chapter4:

Discuss the implementation of the project with diagrams and screenshots, which includes honeypot configurations, data collection, and dashboard design.

Chapter5:

Discuss about the implementation result of collected data, like attack patterns, visualizations, and analysis.

Chapter6:

The conclusion of the interim report which includes the summary of findings from the implementation results and future improvements, etc will be discussed in this chapter.

1.6 Gantt Chart for FYP 1

Activity Week	1	2	3	4	5	6	7	8	9	10	11	12
Project Planning & Setup												
Confirm project title												
Submit project proposal												
Literature Review research												
Prepare Literature Review chapter												
Project Design and Planning												
Define scope and objectives												
Select SDLC model (Waterfall)												
Identify tools & frameworks (T-Pot, ELK)												
System Design & Prototyping												
Plan diagram designs												
Design diagrams												
Create wireframe designs												
Finalization & Submission												
Write interim report conclusion												
Format references, appendix, ToC												

Table 1.1: Gantt Chart FYP 1

1.7 Gantt Chart for FYP 2

Activity Week	1	2	3	4	5	6	7	8	9	10	11	12
System Validation & Data Collection												
Verify Cloud Deployment & T-Pot Logs												
Full-Scale Data Collection (24/7 Runtime)												
Threat Intelligence Development												
Develop Python Enrichment Script (API)												
Implement MITRE ATT&CK Mapping												
Generate JSON Threat Feeds & Testing												
Results Analysis & Dashboarding												
Config Custom Kibana Dashboards												
Analyze Trends & Isolate Top Threats												
Final Reporting & Submission												
Implementation Documentation												
Results Analysis Documentation												
Final Report Review & Formatting												
Final Presentation Preparation												

Table 1.2: Gantt Chart FYP 2

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

This chapter discusses several existing honeypot types and tools that have been implemented which can be seen used by various organizations. Different classifications of honeypots will also be discussed on the logic behind how they work and how it has been implemented. Meanwhile, these mechanisms of the implementation of honeypots are documented and published case studies online, followed by an explanation in detail, also the pros and cons of its implementation. Some innovative ideas were proposed for further improvement. At the end of this chapter, a comparison between these honeypots was also made in different views.

2.2 Honeypot Types and Tools

Before further discussing about honeypot classifications, we need to correctly understand the meaning. Honeypot is a cybersecurity mechanism that uses a manufactured attack to lure cybercriminals away from legitimate targets (Narendran Vaideeswaran et al., 2025). They also gather intelligence about the identity, methods and motivations of adversaries.

2.2.1 Honeypot Classification

Honeypots can be classified or categorized in two ways. One is to differentiate them by their interaction level such as ‘Low-Interaction’ or ‘High-Interaction’, and sometimes even ‘Medium-Interaction’. Each having its own features and functionalities that differ from each other by the mentioned interaction levels. Besides, the other way to classify a honeypot is by its purpose of use. And there are two main purposes of using a honeypot, it is either a research honeypot or a production honeypot.

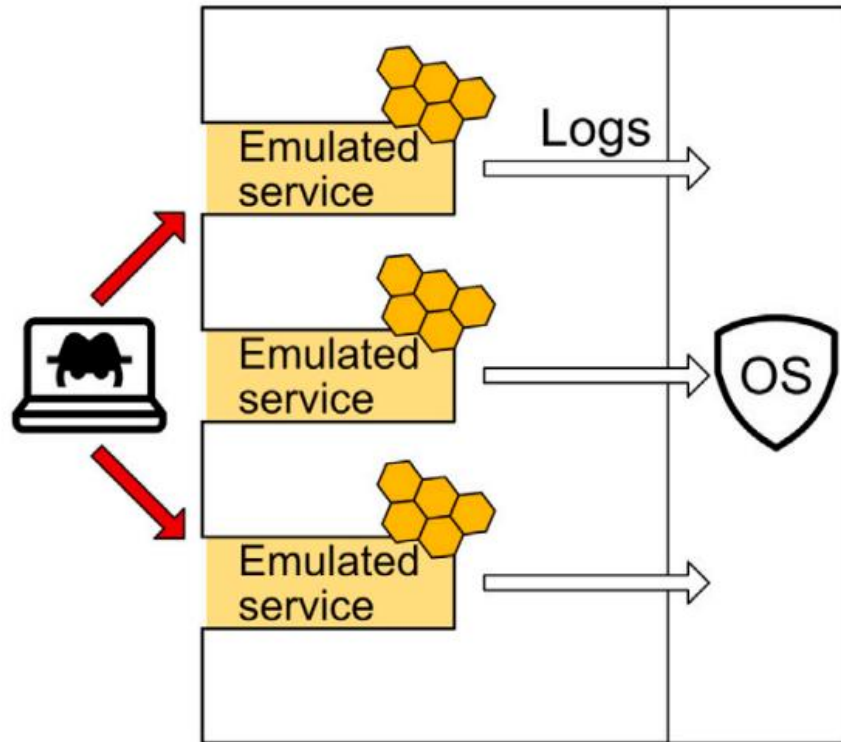


Figure 2.1: Low-Interaction Honeypot

The least amount of interaction is offered to an adversary connecting to the system by low-interaction honeypots (LHs), which give the attacker a limited number of attack paths rather than an operating system (OS) (Ilg et al., 2023). Meaning that attackers are given minimal engagement with the LH, e.g., emulated login screens. Besides, LHs simulate limited services as well such as open ports, basic protocols like HTTP/SMTP and so on. The main purposes of this type of honeypot are to monitor connection attempts and detect credential brute-force attacks.

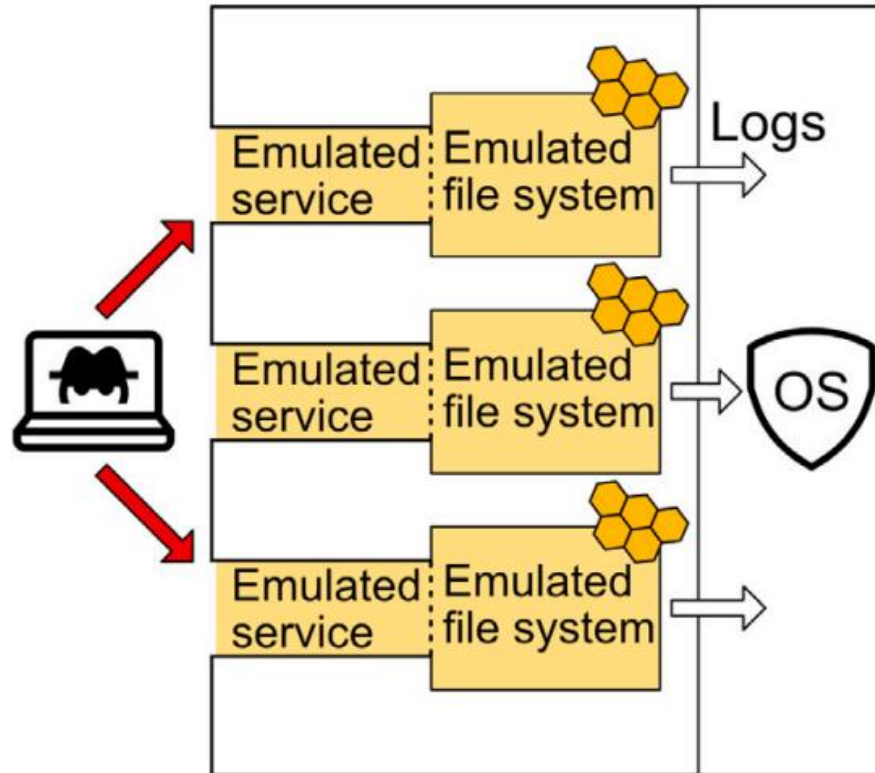


Figure 2.2: Medium-Interaction Honeypot

Medium-interaction Honey pots (MHs) attempt to present a more appealing target and capture a wider range of attacks because they can mimic a system shell to run commands on, but they still do not offer a real operating system (Ilg et al., 2023). Offering partial emulation of services, which allows deeper interaction compared with LHs. For example, MHs can gather malware samples uploaded by the adversary in addition to running specific system commands, and depending on how deep the simulated system is configured. As shown in Figure 2.2, The creation of a deep shell emulation requires a substantial effort because the honeypot application is still isolated from the operating system. Capable of trapping human adversaries for a short period of time.

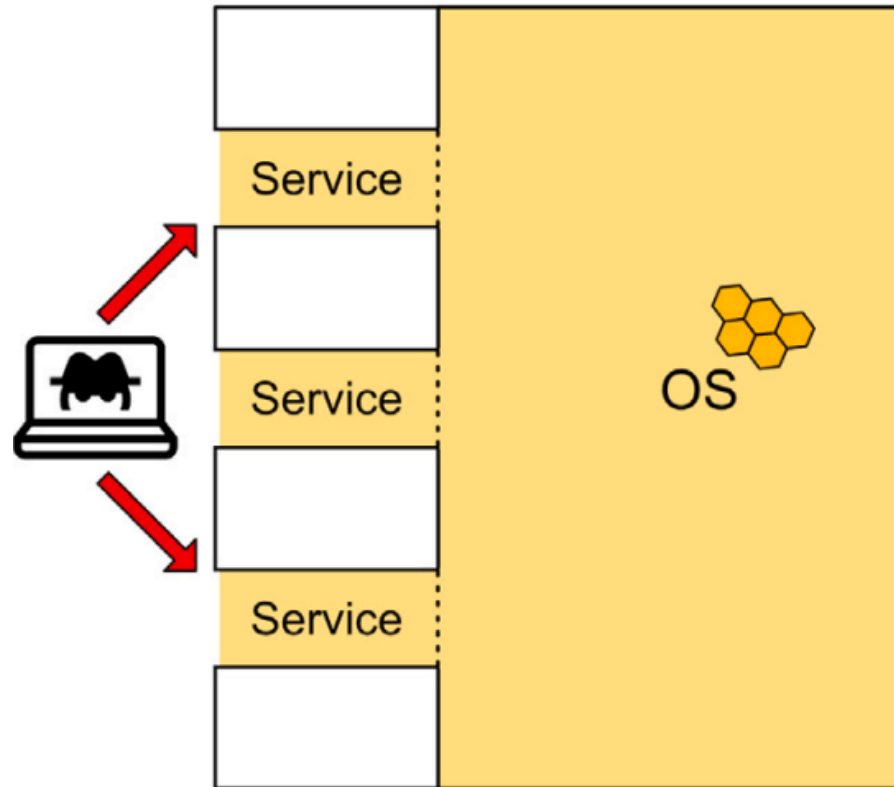


Figure 2.3: High-Interaction Honeypot

As seen in Figure 2.3, they have a real operating system that appears vulnerable to the outside world, therefore high-interaction honeypots (HHs) have the best chance of trapping a human adversary. They also provide insight into the movements and activities of attackers on the system (Ilg et al., 2023). Thus, it introduces risks as well, having HHs to be needing the most maintenance efforts. However, HHs provide the most detailed lifecycle of an attack compared to LHs and MHs.

Table 2.1: Comparison between interaction levels

Feature	Low-Interaction	Medium- Interaction	High-Interaction
Complexity	Low	Moderate	High
Data Depth	Basics (e.g. IP logs)	Payloads, commands	Full attack lifecycle
Risk	Minimal	Moderate	High
Deployment Cost	Low	Moderate	High
Detection Likelihood	High	Moderate	Low

2.2.2 Comparison of Honeypot Tools

Honeypots discussed in this section are in the case of which they were being deployed individually. Each being utilized for its own features and services included for research.

Beginning with **Cowrie** honeypot, it is classified as a medium or high interaction level honeypot. Having it focused on SSH/Telnet emulations with shell interaction logging as well. It is categorized as a high-interaction level honeypot taking into account that it is able to simulate a Debian-like filesystem to mimic a real OS (Knochel & Wefel, 2022). And as a medium-interaction level honeypot due to its feature to log attacker commands such as wget, curl, etc. and also including malware downloads. Moreover, it mainly uses Python virtualization for containment, preventing breakout to host systems.

As for a low interaction level honeypot, **Dionaea** is commonly used as it is focused on capturing malware through SMB, HTTP, and SIP protocols. To explain further on its focus, it emulates vulnerable services like SMBv1 to lure exploit-based attacks. Besides, it also downloads and stores malware payloads such as ransomware and crypto miners. While it has malware intelligence by capturing binaries for sandbox analysis for VirusTotal integration, and able to be deployed on Raspberry Pi or a cloud virtual machine (Agrawal et al., 2022). It has some drawbacks due to its limited interaction depth, attackers are not able to execute post-exploit commands towards Dionaea with a complex configuration for multi-protocol support.

A different approach of a low interaction level honeypot is a honeypot tool named **Honeyd**. It is mainly used for deploying a network-level deception through virtual host simulation. Being able to spoof OS fingerprints like Windows and Linux, to mimic diverse systems. Besides, it simulates thousands of IPs on a single host. It is commonly used for its scalability for large scale IP reputation studies (Manchanda & Tripathy, 2020), and for its support for custom scripting of Lua scripts to emulate service responses.



Figure 2.4: T-Pot Honeyypot

Having mentioned few honeypot tools that are classified as a specific interaction level honeypot, **T-Pot** is quite different from them. It is a multi-honeypot platform that is able to combine different honeypot interaction levels together. It is popularly used these days for its unified threat intelligence with the integration of ELK Stack, which will be further discussed. On top of that, it has more than 20 preconfigured honeypot tools that includes the mentioned Cowrie and Dionaea honeypots. And with the ELK Stack integration, a centralized dashboard (Kibana) is configurable for real-time attack visualization. T-Pot is cloud-ready with the deployment on AWS (Amazon Web Services)/Azure (Sun et al., 2022), and as it was built to juggle multiple honeypot tools simultaneously. Therefore, it has a high resource demand with a considerably steep learning curve for customization.

2.3 Threat Intelligence Methodologies

2.3.1 Log Analysis



Figure 2.5: ELK Stack

ELK Stack (ElasticSearch, LogStash, Kibana) is an open-source toolkit that is popular and used in the centralized log management and threat detection. ElasticSearch is simply a storage and search engine that indexes the processed logs and allows search of huge volumes of data with fast speed. It has a very efficient schema free structure, enabling it to manipulate structured and unstructured data. As an example, the structured data would be firewall logs and the unstructured data would be application error messages. In the case of LogStash, it serves as the pipeline of data, which gathers information on a variety of sources, such as servers, firewalls or applications. Breaking them down into a standard format can screen SSS brute-force attacks on raw logs and label them as high-risk events. Kibana visualization displays all the indexed data in the form of dashboards, heatmap and graphs. It is used by security teams to identify trends such as a sudden increase of malware downloads over port 8080 or geolocation clusters of attacker IPs.

ELK Stack is used to correlate the pattern of attack in the cybersecurity sector. An example that the ELK Stack may be used to identify anomalies in network traffic involves analyzing firewall, router, and server logs in real time (Sabri et al., 2019). Its scalability is appropriate when it is used in large-scale deployments like those in companies like GitHub and Netflix where it monitors millions of events on a daily basis.

Sigma Rules are standardized and open-source detection templates, which outline malicious patterns on logs. They are used in conjunction with ELK Stack to

automate threat hunting. In simple terms, Sigma Rules are attacker strategies such as ransomware encryption or SSH brute force converted into search queries comprehensible by Elasticsearch. An example is that a rule may raise an alarm on frequent failed logins and a successful one as credential-stuffing. It is characterized by the consistency in which the rules can be applicable in other tools such as ELK and Splunk minimizing manual labor. In addition, within a cloud environment, Sigma Rules have the ability to identify suspicious activities such as unauthorized access to AWS S3 bucket by comparing log entries to predetermined rules. In combination with a visualization tool such as ELK Stack, teams can detect and stop attacks fast (Şuştic et al., 2022).

2.3.2 Attack TTPs

THE MITRE ATT&CK MATRIX

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Exfiltration	Command and Control
Drive-by Compromise	CMSTP	Accessibility Features	Access Token Manipulation	Access Token Manipulation	Account Manipulation	Account Discovery	Application Deployment Software	Audio Capture	Automated Exfiltration	Commonly Used Port
Exploit Public-Facing Application	Command-Line Interface	AppCert DLLs	Accessibility Features	BITS Jobs	Brute Force	Application Window Discovery	Distributed Component Object Model	Automated Collection	Data Compressed	Communication Through Removable Media
Hardware Additions	Control Panel Items	AppInit DLLs	AppCert DLLs	Binary Padding	Credential Dumping	Browser Bookmark Discovery	Exploitation of Remote Services	Clipboard Data	Data Encrypted	Connection Proxy
Replication Through Removable Media	Dynamic Data Exchange	Application Shimminig	AppInit DLLs	Bypass User Account Control	Credentials in Files	File and Directory Discovery	Logon Scripts	Data Staged	Data Transfer Size Limits	Custom Command and Control Protocol
Speerphishing Attachment	Execution through API	Authentication Package	Application Shimminig	CMSTP	Credentials in Registry	Network Service Scanning	Pass the Hash	Data from Information Repositories	Exfiltration Over Alternative Protocol	Custom Cryptographic Protocol
Speerphishing Link	Execution through Module Load	BITS Jobs	Bypasses User Account Control	Code Signing	Exploitation for Credential Access	Network Share Discovery	Pass the Ticket	Data from Local System	Exfiltration Over Command and Control Channel	Data Encoding
Speerphishing via Service	Exploitation for Client Execution	Bootkit	DLL Search Order Hijacking	Component Firmware	Forced Authentication	Password Policy Discovery	Remote Desktop Protocol	Data from Network Shared Drive	Exfiltration Over Other Network Medium	Data Obfuscation
Supply Chain Compromise	Graphical User Interface	Browser Extensions	Exploitation for Privilege Escalation	Component Object Model Hijacking	Hooking	Peripheral Device Discovery	Remote File Copy	Data from Removable Media	Exfiltration Over Physical Medium	Domain Fronting
Trusted Relationship	Initial UI	Change Default File Association	Extra Window Memory Injection	Control Panel Items	Input Capture	Permission Groups Discovery	Remote Services	Email Collection	Scheduled Transfer	Fallback Channels
Valid Accounts	LSASS Driver	Component Firmware	File System Permissions Weakness	DCShadow	Kerberoasting	Process Discovery	Replication Through Removable Media	Input Capture		Multi-Stage Channels
	Mahta	Component Object Model Hijacking	Hooking	DLL Search Order Hijacking	LLMNR/NBT-NS Poisoning	Query Registry	Shared Webroot	Man in the Browser		Multi-hop Proxy
	PowerShell	Create Account	Image File Execution Options Injection	DLL Side-Loading	Network Sniffing	Remote System Discovery	Taint Shared Content	Screen Capture		Multi-Brand Communication
	Regsvcs/Regasm	DLL Search Order Hijacking	New Service	Deobfuscate/Decode Files or Information	Password Filter DLL	Security Software Discovery	Third-party Software	Video Capture		Multi-layer Encryption
	Regsvr32	External Remote Services	Path Interception	Disabling Security Tools	Private Keys	System Information Discovery	Windows Admin Shares			Remote Access Tools
	Rundll32	File System Permissions Weakness	Port Monitors	Exploitation for Defense Evasion	Two-Factor Authentication Interception	System Network Configuration Discovery	Windows Remote Management			Remote File Copy
	Scheduled Task	Hidden Files and Directories	Process Injection	Extra Window Memory Injection		System Network Connections Discovery				Standard Application Layer Protocol
				Network Share Connection Removal						
				Obscured Files or Information						
				Privat Modification						
				Port Knocking						
				Process Doppelganging						
				Process Hollowing						
				Process Injection						
				Redundant Access						
				Regsvcs/Regasm						
				Regsvr32						
				Rootkit						
				Rundll32						
				SIP and Trust Provider Hijacking						

Figure 2.6: MITRE ATT&CK Framework Matrix

Although log analysis is an important component in identifying and preventing cyber threats based on the MITRE ATT&CK framework. The security teams can use logs of multiple sources of information to analyze the work of adversaries who have a tactic, techniques and procedures described in the framework. As an example, process monitoring logs, file monitoring logs, and windows event logs are essential sources of data that assists in discovering malicious purpose such as credential dumping or illegal system access. These logs are gathered by tools like Sysmon and Endpoint Detection and Response (EDR) platforms so that they can be mapped by team to specific ATT&CK techniques, like, for example, a Scheduled Task Execution (T1053) or OS Credential Dumping (T1003) (Mustafa Farouk Abo El Rob et al., 2024; Rajesh et al., 2022).

Organizations are also keen on the analysis of logs by concentrating on the most frequently used data sources associated with the ATT&CK techniques. As an example, process command-line parameters and network traffic logs are often utilized in order to identify such techniques as "Lateral Movement" (TA0008) or "Command and Control" (TA0011). It is these logs that are combined and analyzed using Security Information and Event Management (SIEM) tools and benign activities are filtered out to reduce the number of false positives. As reported in the SolarWinds case study, a combination of ATT&CK and SIEM can assist organizations to be aware of sneaky threats i.e. malicious software updates by matching the log patterns with the behaviour of known adversaries (Mustafa Farouk Abo El Rob et al., 2024).

MITRE ATT&CK also helps teams to optimize log collection strategies. As an example, the authentication logs and registry changes that are required by techniques such as "Credential Access" (TA0006) may be monitored, and "Discovery" (TA0007) tactics are based on network sniffing detection based on analyzing encrypted traffic. The matching of log analysis with the structured matrix of ATT&CK enhances the organization to identify high-priority threats and take proactive measures to mitigate them, which contributes to the strong defence of cybersecurity (Rajesh et al., 2022).

2.3.3 Threat Sharing

MISP (Malware Information Sharing Platform) is an open source platform, which is a popular tool that helps organizations exchange cyber threat intelligence (CTI). MISP builds community-based defences to online threats by giving users the ability to share structured threat information (e.g. indicators of compromise (IoCs), attack patterns and tactics of adversaries). As an example, the security teams will be able to provide information on phishing or malware signatures, which will be used by others to prevent attacks even before they happen. MISP is compatible with standardized formats, such as STIX, and compatible with other tools, such as SIEM systems, which makes it simpler to automate threat detection and response processes (Stojkovski et al., 2021).

The main advantage of MISP is that it is decentralized. Companies are able to have their unique instances of MISP and synchronize with others so that it forms a network of trusted communities. This arrangement will allow sharing sensitive information with only authorised partners but still allow large-scale cooperation. In situations such as the SolarWinds compromise, such sharing can enable organizations to reduce risks faster by sharing reports of threats. Non-technical users, however, may not easily adopt the platform since it is not user-friendly and has a steep learning curve. According to a user study, experts like the flexibility of MISP, but beginners become confused because of its activities, a complex interface, and use of numerous terms (Stojkovski et al., 2021).

2.4 Case Studies of Honeypot implementation

The following section will be talking about the comparative case studies which have been discovered by reviewing 7 aspects such as deployment, honeypot types, duration, data analysis method, key finding, unique contribution and the best use case at the end of the chapter.

2.4.1 Case Study 1

In the study by Agrawal et al. (2022), honeypots were deployed extensively and over a long period in an open cloud-based setting to examine the behavior of adversaries in various levels of interaction. Their study was a continuous process of eleven months and it was one of the longest studies in terms of time. The honeypot infrastructure was implemented on the Microsoft Azure which comprised of five low-interaction honeypots (LIHPs) and one medium-interaction honeypot (MIHP). These honeypots were set up with containerized environments and put on the Azure Web App services. The honeypots all pretended to be vulnerable systems exposed to the Internet, and LIHPs ran like a static blog-like web page whereas MIHPs looked like a dynamically-run content management system with an administration backend vulnerable to attacks involving credential usage. The subsection title must not start in a last line of a page. The key objective of the research was to get to know how attackers exhibit different behaviors with regard to the types of honeypots. The team has included an effective logging and data security solution, which redirects all the logs on the interaction to an unalterable data store within an isolated data warehouse in Azure Data Lake. The behavior of the system was simulated by using real time handlers and when attackers issued commands like wget or tried to make SQL injection through tools like sqlmap. Not only did these handlers assist in capturing the interaction between the attackers, but also enabled a more in-depth interaction, which allowed the adversaries to think they have compromised a real system.

Text Based on the results of the data, the researchers noted that there were considerable differences in the traffic volume and the level of attacks in the honeypots. The most frequent attacks occurred with the LIHPs, with the most common being LIHP-4 with more than 31,000 interaction sessions and 78 content uploads. The MIHP,

on the contrary, had 138 interaction sessions and zero content uploads. Such an imbalance highlights the fact that attackers prefer to target easier and simpler targets. Latent Dirichlet Allocation (LDA) which is a machine learning method, was also used to further analyse the logs to extract thematic clusters of the interaction content. The review showed that the majority of the adversarial traffic was automated and located all over the world, with a significant percentage of the traffic coming in form of Brazil, the United States, China, and Russia.

The paper identified some important trends in the behaviour of attackers. Markedly, attackers were more active in the evening (according to the time of the data center), and the majority of the interactions did not last long, with the interactions being one-sided in most cases. Posts were mostly made of mal inspiring scripts and attempt of credential brute-force bot attacks. The analysis of the request types and payloads was another especially new input of the case study, with most of the requests being GET (more than 90 percent), then POST and then HEAD. The topic modeling was also used by the researchers to isolate clean request patterns by raw logs and found common exploits with PHP systems and network scanning patterns (e.g. Nmap signatures).

The implementation of cloud-based honeypots by Agrawal et al. has proven the practice to be viable and efficient in its collection of long-term threat intelligence. They conclude that low-interaction honeypots are still useful in monitoring some wide scale automated attacks and the medium-interaction systems are not fully used unless deliberately targeted. Notably, the research confirms the applicability of LDA and log segmentation to the classification of threats to support real-time monitors and automatic response controls (Agrawal et al., 2022).

2.4.2 Case Study 2

Ilg et al. (2023) presented a survey of the architecture, deployment systems, and practical use of open-source honeypots in the enterprise IT. The fact that their work is not only characterized by the richness of the technical investigation, but also by the practice of its deployment is important. The authors have conducted a review of more than a dozen honeypots by their level of interaction (low, medium and high) and evaluated them by the level of architectural sophistication of the honeypots, logging, protocol coverage and resistance to fingerprints. The three most interesting honeypots which were studied were Cowrie (medium-interaction SSH/Telnet emulator), Dionaea (low-interaction malware collector), and Wetland (high-interaction Docker-based SSH honeypot).

The authors began their study by setting each honeypot in controlled testbeds to test its behavioral realism, fingerprintability and ease of deployment. This practical tactic enabled them to contrast actual reaction patterns and interplay of the attackers. They emphasized that honeypots which have high interaction such as Wetland are especially efficient in attracting advanced human attackers. Wetland is a Docker environment solution that allows the emulation of most of the system and offers effective logging and attacker isolation. Low-interaction honeypots like Heralding and Glutton, on the other hand, were complimented by their low installation requirements and their capability to provide multiple protocols at the same time.

A very large part of the research was devoted to avoidance methods and the danger of honeypot detection. The authors have shown how default settings and fixed SSL/TLS fingerprints of certain honeypots may result in their easy detection by software such as Shodan or a timing attack. They also talked about passive and active fingerprinting, where frequent changes in configuration were important and diversity in deployment to maintain honeypot stealth. As an example, they have observed that a lot of honeypots have old default certificates, or even respond unrealistically quickly to a request, which is easy to spot by an adversary.

The other significant point of focus was deployment flexibility. The article gave the in-depth comparisons between containerized and bare-metal deployment.

Containerization especially with Docker and Kubernetes was identified to make management and scaling easier, however, they can provide distinct time and network artifacts that can be identified by experienced attackers. Docker images and deployment tutorials as well as options of configuring popular honeypots are also listed by the authors, so the work can be regarded as an invaluable source of data to practitioners.

Finally, the Ilg et al. survey is not merely a list of tools but provides strategic recommendations on the choice of honeypots and their implementation depending on the purpose of each operational activity. Whether the aim is to gather malware, brute-force attack, or learn human adversarial behavior, their analysis gives a roadmap that is based on the level of interaction, risk of fingerprint, and overhead. Their results are a solid argument in favor of the application of multi-honeypots with different architectures to achieve the highest visibility of threats and the lowest detectability risk (Ilg et al., 2023).

2.4.3 Case Study 3

The article, which is called Advancing Cybersecurity with Honeypots (2025), conducts a comparative analysis of various popular honeypots, such as Honeyd, Cowrie, Dionaea, Glastopf, and Kippo. This research did not concentrate on one deployment environment but it was a functional benchmarking exercise over a standardized testbed. The objective was to test each honeypot through its detection performance, emulation realism, configuration complexity, logging granularity and versatility to the various network topologies.

The authors started by categorizing the honeypots on three interaction levels and organized the assessment as such. All tools were implemented in the same network and hardware environment in order to be consistent. A major parameter under consideration was the number of supported protocols-Dionaea came out as powerful in malware collection since it employs services such as SMB, FTP and HTTP protocols whereas Cowrie was quite effective in capturing attacker commands within an emulated SSH setting. Honeyd was known to have the capability of generating the whole topology of the network by the use of templates, and that it was suitable in large-scale deception.

Other evaluation criteria included realism and fingerprint resistance. According to the authors, Kippo and Cowrie, though successful in capturing the SSH brute-force attacks, have drawbacks in their predictable file structure and pattern of responding. On the other hand, protocol diversity was found lacking in Glastopf which emulated better content but was found to mimic vulnerable web servers. The qualitative matrix was used to score each honeypot based on set up time, extensibility and community support. The significant contribution of this research was the recommendation matrix in which the honeypots were correlated with security objectives. As an example, Dionaea was recommended to be used by organizations interested in gathering and analyzing malware samples, whereas Cowrie or Wetland was recommended to those interested in attacker behavior and TTPs (Tactics, Techniques, and Procedures). Another recommendation that the authors put forward is the implementation of hybrid deployment models- high-interaction and low-interaction honeypots together in layered visibility.

The operational focus of the case study is the most valuable thing about it. The researchers have been practical in terms of administering the administrative overhead of each honeypot deployment and maintenance. As an example, Honeyd was mentioned to be difficult to configure but highly specialized, but Cowrie was easy to learn and well supported by the community. These lessons aid in informing deployment choices in the enterprise and academic settings.

To sum up, comparative assessment can play an important role in the selection of honeypots in strategic terms to collect threat intelligence. This work provides practical ideas on how to create resilient and adaptable honeypot infrastructures by overcoming the trade-offs between fidelity, resource consumption, and ease of use (Rajesh et al., 2025).

2.4.4 Comparison between different honeypot deployments case study

Table 2.2: Contrast between case studies

Feature	Case Study 1	Case Study 2	Case Study 3
Deployment	Azure Cloud VMs	Local/Cloud via containers	Multi-platform, Simulated environments
Honeypot Types	LH & MH	LH, MH, HH (e.g. Wetland, Cowrie)	LH, MH, HH (e.g. Kippo, Dionaea)
Duration	11 Months	Survey-based	Short-term simulated attacks
Data Analysis Method	LDA Topic Modelling	Architectural analysis	Qualitative performance matrix
Key Findings	Most attacks hit LH only	HHs best for human attackers	Trade-offs in detection vs. usability
Unique Contribution	Long-term, real cloud environment	Detection evasion, container use	Operational benchmarks for comparison
Best Use Case	Trend analysis in real cloud	Framework/tool selection	Tool evaluation and selection

CHAPTER 3

METHODOLOGY

3.1 Overview

The details of methodology are explained in this chapter, such as system development life cycle, hardware, and software requirements. All of these components are considered extremely important to smoothly deploy the honeypot and for threat intelligence analysis. It is the guideline to the developer. The idea of a project can be transformed into a working and functional structure when the SDLC phases are strictly followed. For this project, the model that was implemented was called the Waterfall model. This model consists of seven stages which will be explained later.

3.2 Waterfall model

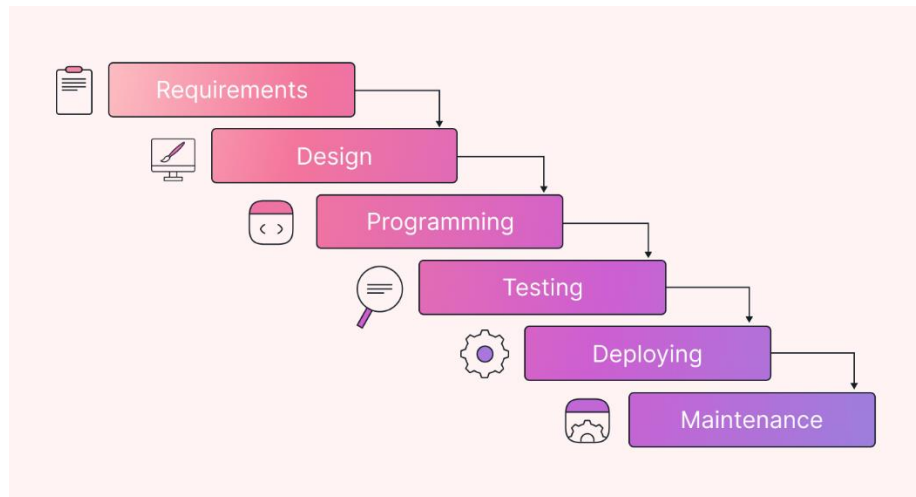


Figure 3.1: The phases of the waterfall model

The reasons for using the waterfall model are as below:

- Clear structure with the sequence of phases

By using the waterfall model, the deployment process was divided into several phases. Therefore, the design process will be linear, which means the previous stage must be completed before the next stage comes in. Besides, if any problems are encountered in that specific phase, they can also make sure the task to be done will not overlap.

- The end goal is determined early

The waterfall model also allows us to determine the output that we would like to achieve at the very beginning of the phase. Since the goal is set earlier, the developer can focus on the current task before moving to the next stage. For this project, the end goal is to deploy a working multi or hybrid honeypot that is able to collect attack vectors' activity for threat intelligence and sharing. The possibility of getting lost as a developer will also be reduced.

- Suitable for a smaller project

A small project like a honeypot deployment is very suitable to implement the waterfall model as the output and requirement of this project are fixed. The requirement of the output is specific, unlike the agile model which can be more complex and would require a lot of changes because of the customer's involvement.

3.2.1 The phases of the waterfall model

The Waterfall model consists of seven phases which are requirements, system analysis, system design, implementation, testing, deployment, and maintenance. The details of the seven phases of the waterfall model are explained below:

3.2.1.1 Requirement

The very first phase of the system development life cycle is a requirement. As a developer of this honeypot, it was necessary to understand what needs to design, the purpose, and the functions required in this project. Therefore, several steps such as doing some research and making a literature review on the existing similar honeypot deployments are compulsory. By doing this, we can have a clearer picture of the problems, resulting in having more ideas to improve and enhance the project to make it more useful.

In implementing this phase, the concepts from structured analysis are applied to identify the scope and deliverables. Tools like **use case modelling** are used to define how the system will interact with attackers and which behaviors we expect to log. Interviews and observations based on attacker behavior studies from selected case studies guide the construction of a realistic threat scenario. Specific attention is paid to differentiating the needs between low-interaction and medium-interaction honeypots.

3.2.1.2 Analysis

The second phase is analysis, in this phase, we identified the problems that have been found along the way, which will then come out with the problem statement. The problem statement greatly assists us to identify why this project is needed. According to the literature review that has been made, the honeypot that has the most integration tools and features is T-Pot, while other honeypots only offer specific aspects of data depth, providing a lesser attack in-depth life cycle to be analyzed.

During implementation, this phase utilizes requirement classification strategies such as **root cause analysis** and **outcome analysis** to differentiate between technical limitations and true system needs. We apply the concept of business rules definition, ensuring that our project captures behaviors aligned with the **MITRE ATT&CK framework**. This enables the output of this phase to directly inform the architecture and configuration of the honeypot services.

3.2.1.3 Design

The third phase of the waterfall model is the system design. After the requirement and analysis are made, the overall system structure needed to be designed to provide a clearer view of how the system will be implemented which is in phase five. System design can be divided into two categories which are logical design and physical design. The logical design is concept-oriented which shows how the system works to the users, the users would learn and know how the application works like a user manual. The physical design was then produced where all the command lines and configuration codes are stored.

In practice, this phase involves drafting architecture diagrams that include the network topology, data flow between components (honeypots, Logstash, Elasticsearch, Kibana), and security zoning. We also conduct evaluation of interaction levels (LIHP/MIHP) and design Docker container mappings for each service.

3.2.1.4 Implementation

The fourth phase is the implementation phase, which involves putting the project plan into action. All the requirements that was being made in phase one were met. The system design was translated into source code or configuration code where a working system is output. The functions which are proposed in the system design should also be included and implemented in the system. In this project, honeypot functions such as Kibana dashboard, ElasticSearch logs, and Grafana attack trends display will be configured inside the honeypot.

During this phase, the **T-Pot honeypot system** is deployed using Docker. Step-by-step scripting and environment configuration are executed, including port mapping and honeypot-specific configuration files. Implementation also includes log aggregation scripts, timestamp synchronization across containers, and validation of proper functioning of

individual services. Real-world simulation of attack vectors is conducted to verify the responsiveness of each honeypot.

3.2.1.5 Testing

The fifth phase of the waterfall model is testing. At this phase, the honeypot implementation was tested out from the beginning to the end. For instance, the deployment, the console connectivity, validating responsiveness by simulating attacks, ensure logs are captured in Elastic, visualizations in Kibana, honeypot isolation from main network, Logstash filters optimizations, reducing processing latency. By doing this, we could identify the possible bugs that may be encountered and make necessary amendments to the configurations.

This phase adopts both **unit testing** (e.g., testing each honeypot container's response to crafted traffic) and **system testing** (e.g., end-to-end log flow verification). Penetration testing tools like Nmap and Hydra are used in a controlled lab setting to simulate attacks. Logs are validated against expected event flows in Elasticsearch and visualization integrity in Kibana. **Acceptance tests** involve validating that the system supports the data needed for threat intelligence analysis.

3.2.1.6 Deployment

The sixth phase of the waterfall model was to deploy the honeypot by migrating the tested setup to AWS for 24/7 availability. Also configuring AWS security group to expose only honeypot ports. Before migrating, all the functions of the honeypot should be working with lesser bugs. At this phase, we observed the behaviour of the honeypot and see if there is any update needed to be made in the future.

Deployment in this context follows DevOps best practices, using automated provisioning scripts for infrastructure on AWS. Security Group rules are explicitly defined to expose only honeypot interfaces. Log forwarding is configured to a secure, immutable S3 bucket or a centralized log server.

CloudWatch is used to monitor resource usage and trigger alerts in case of service failure or suspicious behavior.

3.2.1.7 Maintenance

The last phase of the waterfall model is maintenance, since the honeypot is deployed with full availability, there will be noticeable results of the honeypot implementation. We observed the behaviour of the honeypot and see if there is any update needed to be made in the future. By doing this, it is able to keep the honeypot useful for threat intelligence.

Maintenance includes regular updates to honeypot images, rotation of exposed ports to avoid blacklist bans, and periodic cleansing of old logs. Based on attacker trends, the honeypot services may be reconfigured to mimic more attractive targets. Weekly analysis reports are generated using Kibana dashboards and are reviewed to determine whether attack techniques evolve and require system adaptation.

3.3 Software/tools Required

In this section, the software and tools to deploy and configure the honeypot will be discussed. Various software tools and platforms were utilized to successfully develop, deploy, and analyze honeypot systems. These tools cover honeypot deployment, data logging, traffic visualization, and threat analysis.

3.3.1 T-Pot Honeypot Framework

T-Pot is an open-source all-in-one honeypot platform that integrates multiple honeypots like Cowrie, Dionaea, and Suricata. It is deployed using Docker containers and provides comprehensive logging and visualization support via ELK stack. T-Pot was chosen due to its scalability, community support, and wide protocol coverage.

3.3.2 Docker and Docker Compose

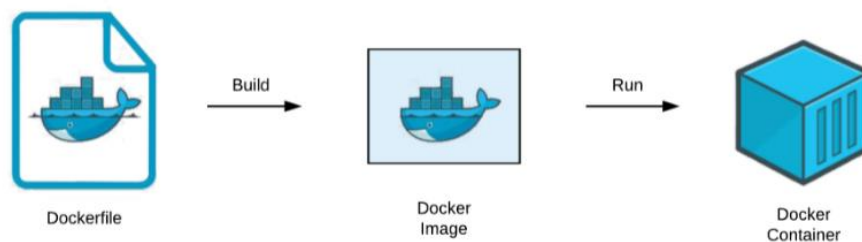


Figure 3.2: Docker and Docker Compose

Docker is used to containerize the honeypots in an isolated environment to improve scalability and deployment speed. Docker Compose allows the configuration and management of multiple services, such as honeypots, log shippers, and dashboards.

3.3.3 ElasticSearch, LogStash, and Kibana (ELK Stack)

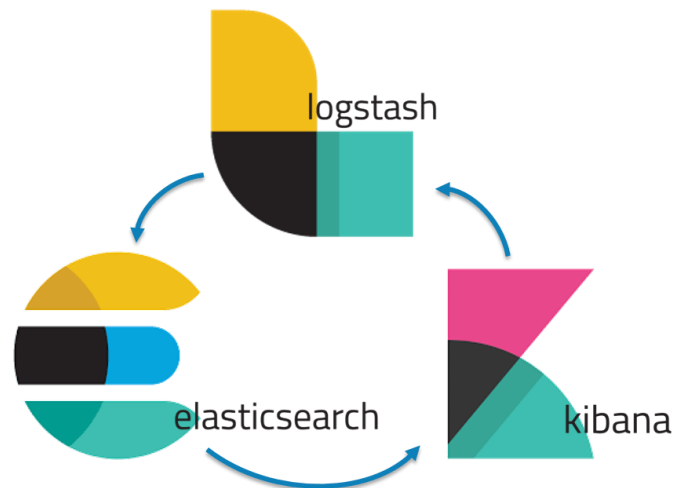


Figure 3.3: ELK Stack (ElasticSearch, LogStash and Kibana)

The ELK stack was employed for centralized logging and visualization of attack data. Logstash collects and parses the logs from honeypots, Elasticsearch indexes the data, and Kibana provides a web-based interface for visualizing attacker behaviour.

3.3.4 Wireshark

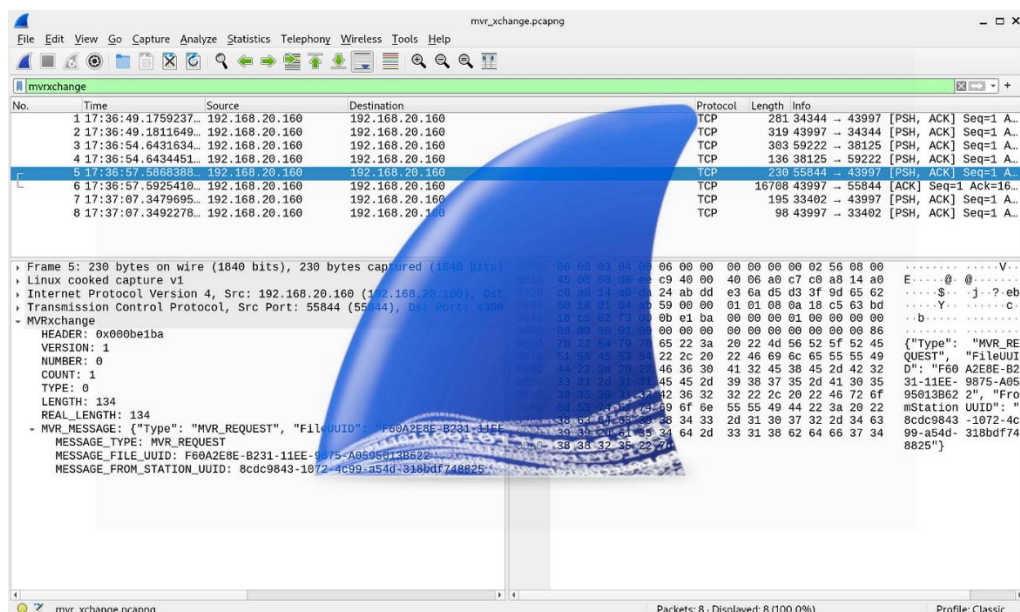


Figure 3.4: Wireshark Interface

Wireshark is a network packet analyzer used to capture and analyze network traffic to and from the honeypot. It aids in understanding the techniques and tools used by attackers during their interaction with the honeypot.

3.3.5 MITRE ATT&CK Navigator

Initial Access	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	Discovery	Lateral Movement	Collection	Command And Control	Exfiltration	Impact
11 items	27 items	42 items	21 items	57 items	16 items	22 items	15 items	13 items	21 items	9 items	14 items
Supply Chain Compromise	Control Panel Items	Security Support Provider	Access Token Manipulation	Access Token Manipulation	Input Capture	Process Policy	Logon Scripts	Input Capture	Domain Fronting	Data Compressed	Endpoint Denial of Service
Drive-by Compromise	PowerShell	AppCert DLLs	Extra Window Memory Injection	Control Panel Items	Credential Dumping	Score: 5 Metadata: -Applicable to: client endpoints -Detection score: 4 -Overlay: Detection	Pass the Hash	Data from Network Shared Drive	Uncommonly Used Port	Data Encrypted	Network Denial of Service
Spearphishing Attachment	Regsvr32	Logon Scripts	Extra Window Memory Injection	Extra Window Memory Injection	Credentials in Registry		Application Deployment Software	Final Collection	Remote Access Tools	Exfiltration Over Command and Control Channel	Data Encrypted for Impact
Exploit Public-Facing Application	Rundll32	Image File Execution Options Injection	Process Injection	Masquerading	LLMNR/NB-NS Poisoning and Relay	System Owner/User Discovery	Distributed Component Object Model	Audio Capture	Commonly Used Port		
External Remote Services	Scripting	Application Shimming	AppCert DLLs	Process Injection	Regsvr32	Account Manipulation	Automated Collection	Clipboard Data	Data Obfuscation	Automated Exfiltration	Destruction
Hardware Additions	Scheduled Task	Image File Execution Options Injection	Image File Execution Options Injection	Rundll32	Brute Force	Process Discovery	Exploitation of Remote Services	Standard Application Layer Protocol	Exfiltration Over Alternative Protocol	Defacement	
Replication Through Removable Media	User Execution	Scheduled Task	Image File Execution Options Injection	Scripting	Credentials in Files	System Network Configuration Discovery	Pass the Ticket	Data from Information Repositories	Communication Through Removable Media	Disk Content Wipe	Disk Structure Wipe
Dynamic Data Exchange	Command-Line Interface	Account Manipulation	Application Shimming	Image File Execution Options Injection	Exploitation for Credential Access	Application Window Discovery	Remote Desktop Protocol	Data from Local System	Connection Proxy	Exfiltration Over Other Channel	Firmware Corruption
Spearphishing Link	Compiled HTML File	Appinit DLLs	Scheduled Task	Timestamp	Obfuscated Files or Information	Browser Bookmark Discovery	Remote File Copy	Data from Removable Media	Custom Contr		
Spearphishing via Service	Dynamic Data Exchange	Authentication Package	Accessability Features	Binary Padding	Forced Authentication	Domain Trust Discovery	Remote Services	Man in the Browser	Screen Capture		
Trusted Relationship	Execution through API	BITS Jobs	Appinit DLLs	BITS Jobs	Hooking	File and Directory Discovery	Replication Through Removable Media	Data Staged	Custc Crypt Proto		
Valid Accounts	Execution through Module Load	Bootkit	Bypass User Account Control	Bypass User Account Control	Input Prompt	Network Service Scanning	Network Share Discovery	Shared Webroot	Man in the Browser		
	Exploitation for Client Execution	Browser Extensions	DLL Search Order Hijacking	Code Signing	Kerberoasting	Network Sniffing	Network Sniffing	Video Capture	Data		
	Graphical User Interface	Change Default File Association	Exploitation for Privilege Escalation	Compile After Delivery	Password Filter DLL	Peripheral Device Discovery	Taint Shared Content		Domz Gene Algor		
	InstallUtil	Component Firmware	Compiled HTML File	Compiled HTML File	Private Keys				Fallb		

Figure 3.5: MITRE ATT&CK Navigator

This tool was used to map observed adversarial techniques to the MITRE ATT&CK framework. IT helps classify and label the tactics, techniques, and procedures (TTPs) based on log data analysis.

3.3.6 Ubuntu 22.04 LTS



Figure 3.6: Ubuntu

The honeypot systems were hosted on Ubuntu 22.04 LTS, which is a secure, stable, and widely supported Linux distribution. It was selected due to compatibility with most open-source security tools and Docker.

3.3.7 VirtualBox (for testing)



Figure 3.7: Oracle Virtual

For offline testing and simulations, Oracle VirtualBox was used to host virtual environments of honeypots before actual deployment. It enables controlled testing without affecting real infrastructure.

3.4 Hardware Required

In this section, detailing the hardware components used to deploy and test the honeypot systems. Two configurations were used: a local machine for testing and a cloud hosted setup for full-scale deployment.

3.4.1 Local Workstation

A Lenovo IdeaPad Slim 3 laptop was used during the development and local testing phases. The specifications are as below:

- Processor: AMD Ryzen 5 5625U (Quad-Core)
- RAM: 16GB DDR4
- Storage: 512GB NVMe SSD
- Network: No on-board Ethernet
- OS: Windows 11

This setup is sufficient to run Docker containers for local honeypot testing and ELK stack visualization.

3.4.2 Cloud Virtual Machine (DigitalOcean/AWS)

For live deployment, a DigitalOcean VM was configured with the following specifications:

- vCPU: 2-core (Standard B2s)
- RAM: 8GB
- Storage: 160GB SSD (Standard-tier)
- Operating System: Ubuntu 22.04 LTS
- Public IP: Dynamic (to simulate typical attack vectors)

The cloud-based deployment provided 24/7 uptime and allow the honeypots to be reachable globally for better attack data collection.

CHAPTER 4

SYSTEM DESIGN

4.1 Overview

The chapter elucidates the design blue print and system level design on the implementation of a multi-honeypot system to threat intelligence. This chapter aims at offering a clear picture of the workflow, architectural, and data handling mechanisms that were used in the system. The diagrams are provided to describe the process through which attackers will interact with the honeypot in visual form and through which the logs will pass through the system to generate threat intelligence and visualize it. Besides the technical diagrams, this chapter contains wireframe prototypes to the Kibana dashboard and data visualization interface, demonstrating how the user interacts and comprehends captured threat data. It has presented the technical implementation of the system, which was at the core but the diagrams and illustrations have been provided to understand better. The system is aimed at implementing a multi-honeypot setup with the help of T-Pot honeypot framework which consists of a combination of honeypots including Cowrie and Dionaea, and ELK Stack to visualize logs in real-time. The implementation plan is tailored to make sure that the deployment is used to capture malicious activity and give threat intelligence insights using centralized analysis dashboards. The diagrams provided are the system architecture, deployment workflow, data flow and use case diagram.

4.2 System Architecture Diagram

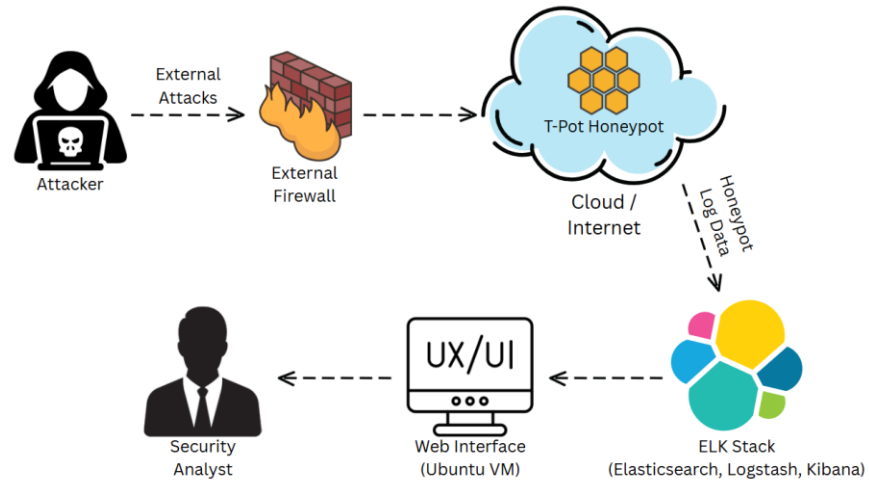


Figure 4.1: System Architecture Diagram

The system architecture diagram presents the overall high level outlay of the honeypot deployment and shows how the components interrelate. It contains Cowrie and Dionaea honeypots that are deployed within Docker containers supplied by T-Pot framework. The honeypots produce logs that are processed by the Logstash and indexed by the Elastic search engine and visualized by the Kibana. The system deployed in this project was in a virtual machine running on Microsoft Azure or Amazon Web Services in order to continuously collect data.

The system architecture diagram gives a picture representation of the whole deployment structure of the T-Pot honeypot system in AWS. It demonstrates the ways of external attackers to interrelate with a virtualized cloud infrastructure to recreate and capture malicious actions. The design starts with Internet-based attackers that attack services located in an Amazon EC2 instance that has been set up with T-Pot. These services contain several containerized honeypots, e.g. Cowrie (to emulate SSH) and Dionaea (to emulate malware interaction on SMB ports and HTTP ports). First, traffic is filtered using AWS Security Groups and possibly AWS WAF (Web Application Firewall) rules in order to permit only explicit ports and protocols.

When the bad traffic is directed to the honeypots, logs are recorded. The logs are automatically fed into Logstash, which splits, formats and sends it to Elasticsearch to

be indexed and stored. Elasticsearch allows searching through data very fast and at scale thus the analysis of the data can be done in real time. All the data is exposed with the help of Kibana, the primary visualization tool used to monitor trends and log entries and their relation to tactics known by the MITRE ATT&CK mappings. The characteristics of this architectural model are that it generates modularity, the ability to see the actions of attackers, as well as the ability to deploy on the cloud in a safe manner, making it a strong system of threat intelligence collection.

4.3 Deployment Workflow Diagram

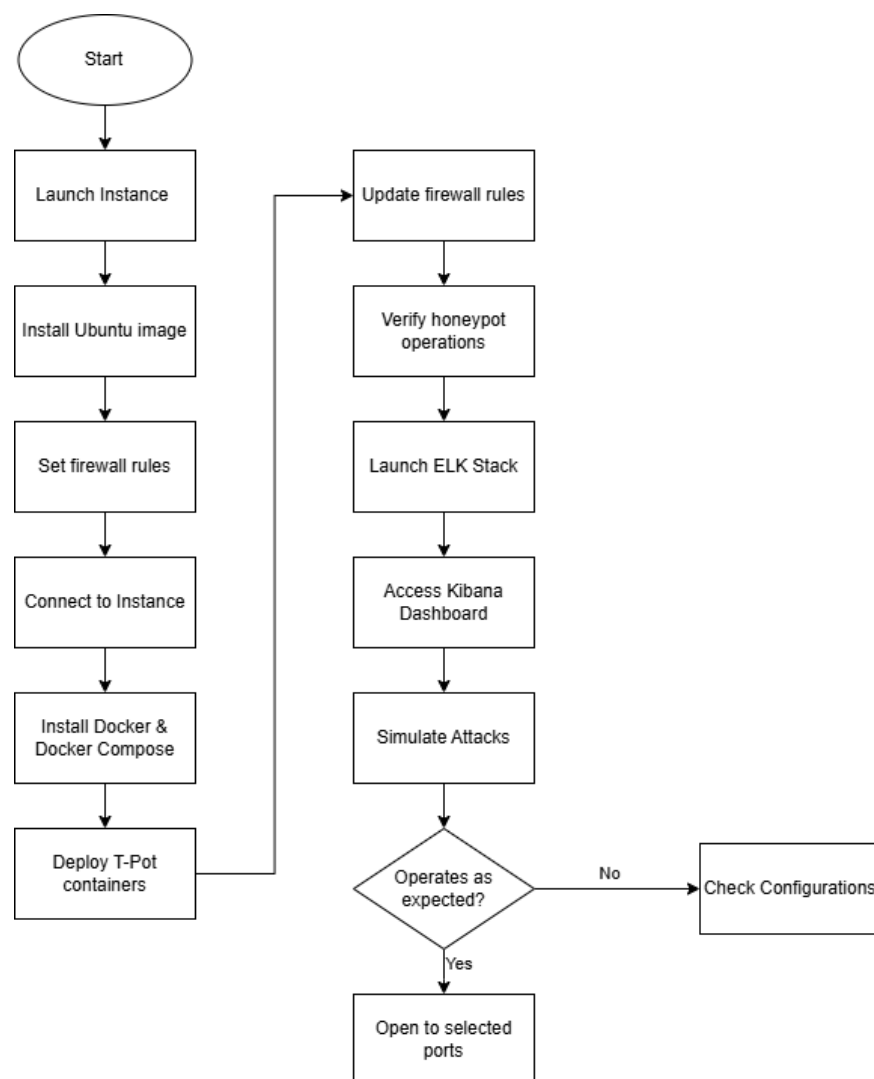


Figure 4.2: Deployment Workflow Diagram

Deployment workflow gives the sequence of steps that need to be followed to deploy the honeypot and other supporting services. It makes sure that every tool is properly set up, tested, and equated in the environment.

The deployment workflow diagram is a step-by-step approach that is followed to configure the T-Pot honeypot environment in AWS. It focuses on the action in sequence as well as the dependencies amongst components. The initial step requires the provisioning of a clean environment of Ubuntu 22.04 LTS with or without AWS EC2. This example uses Docker and Docker Compose that are a requirement before installing T-Pot.

Then, there are some additional steps, such as downloading and starting the deployment scripts of T-Pot that calls the implementation of its containerized honeypot services. These are Cowrie, Dionaeas among others integrated into a secure and monitored system. After deployment, the user sets up firewall rules (e.g., AWS Security Groups) to allow certain ports to each service and block any unneeded traffic. Logstash, Elasticsearch and Kibana are all enabled simultaneously in an effort to process and present the data taken. The last stages are to test the deployment by simulated basic attacks (e.g., SSH brute-force with Hydra), ensure the presence of log collection, and ensure that the dashboards in Kibana are displaying live data correctly. The workflow gives a systematic guide to the deployment of a high-fidelity honeypot system in a reproducible manner.

4.4 Data Flow Diagram

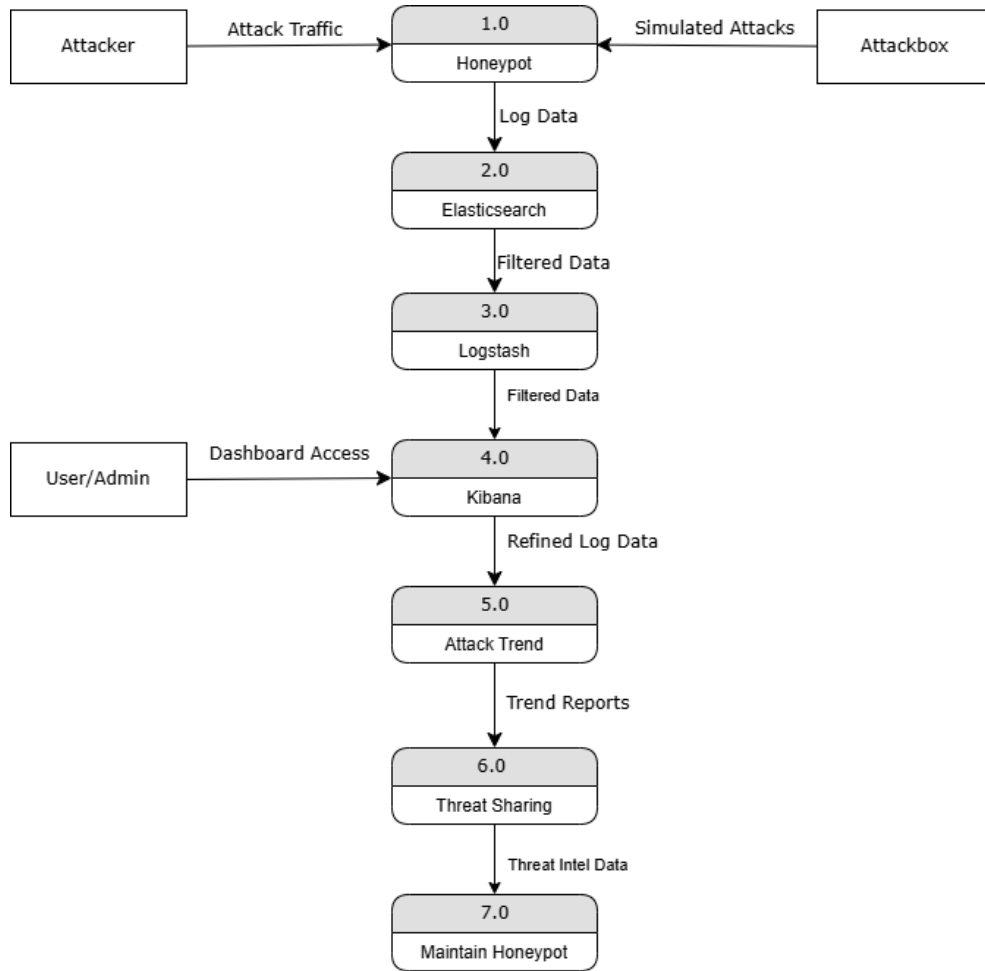


Figure 4.3: Data Flow Diagram

This section displays the data entry starting with the interaction of the attacker to the ultimate analysis phase. It explains the capturing, transferring, processing and visualization of data.

The diagram depicts the flow of information logically in the point of interactivity by the attacker to the ultimate visualization by the analyst. The hacker starts a network connection to services simulated by Cowrie or Dionaea. These honeypots serve as data sources, capturing all the details of the interactions, i.e. commands, payloads and behavior patterns. These logs are sent to Logstash that does the conversion of the unstructured data into structured outputs of the form of JSON.

Logstash filters and assigns time stamps and message parsing and then logs are sent to Elasticsearch that indexes the data effectively to be queried. Elasticsearch is the back-end database in the system. Over this, Kibana is used as the front end interface that makes queries to Elasticsearch and presents the results in human readable dashboards. These are pie charts, geo-location maps, graphs and log viewers. This flow of data keeps all interactions of attackers traceable between the input and the output to facilitate forensics and strategic decision making.

4.5 Use Case Diagram

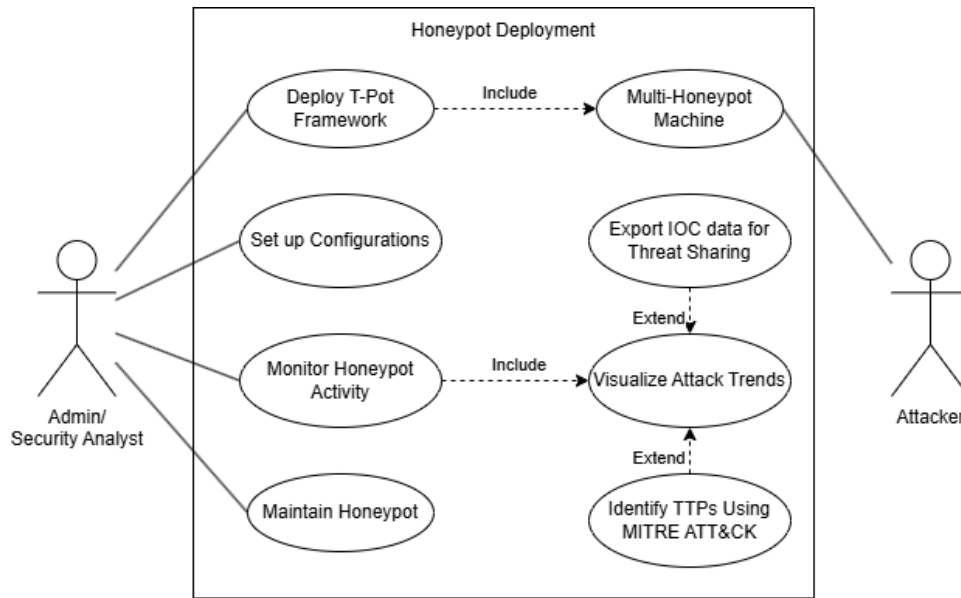


Figure 4.4: Use Case Diagram

The use case diagram illustrates the interactions between two actors of the system, Admin (Security Analyst) and Attacker (passive system user). The attacker indirectly communicates with the system through probing network services inducing logging and response by the honeypot infrastructure. The admin, in his/her turn, has the direct responsibility to configure honeypots, check dashboards, log files analysis, and event mapping to the MITRE ATT&CK matrix.

Applications are installation of the T-Pot honeypot system, viewing Kibana dashboards, sifting data according to attacker signatures, and exporting indicators of compromise (IOCs) to other parties. The version of Admin is associated with each of these use cases to important components of the system, including Docker containers, Kibana, and Elasticsearch. This chart highlights the two-sidedness of system contact: adversarial (through attackers), and analytical (through defenders), and demonstrates the extent of control and monitoring that the administrator has.

4.6 Wireframe Design

A Lenovo IdeaPad Slim 3 laptop was used during the development and local testing phases. The specifications are as below: The wireframe design of this honeypot is shown as below. Providing a brief view of how will the dashboard and interface of the honeypot would be.

4.6.1 Dashboard Overview



Figure 4.5: Wireframe – Dashboard Home Overview

Just as shown in Figure 4.5, the interface is accessible through a website once deployed. Into the dashboard interface, it serves a one-stop web page for user to view all the real-time ongoing events occurring towards the deployed honeypot via visualization of Kibana graphs and charts. Also to mention, a honeypot attack map to have a better picture of the source of the occurring attacks. The dashboard wireframe outlines the initial interface a security analyst encounters when logging into Kibana. The layout features multiple real-time visualization panels including a line graph tracking attacks over time, a pie chart representing protocol usage, and a world map displaying attacker origin by IP geo-location. There's a left-side navigation menu offering access to different dashboard views such as detailed logs, and filter settings.

This wireframe is designed to help users understand attacker behavior patterns at a glance. The real-time capability enables quick assessment of attack frequency and intensity, while filtering tools allow the analyst to narrow down specific threat vectors. The design reflects the actual Kibana UI but simplifies it for clarity in planning. It serves as a prototype for how a fully operational dashboard might look once live data is flowing into the system.

4.6.2 Log Drilldown View

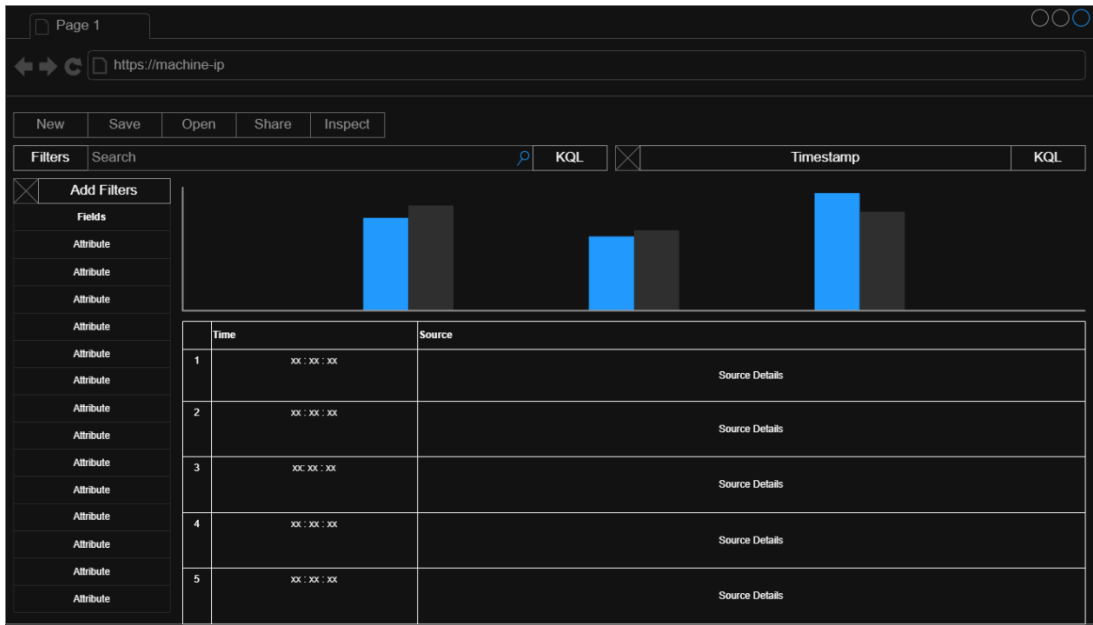


Figure 4.6: Wireframe – Log Drilldown View

In Figure 4.6, it is the wireframe of the logs view via Elastic, having a clear view of the timestamps graph of events at the top and detailed logs of events displaying below of the graph which can be exported for further analysis. Having easy accessible filters to add on the left side panel, allowing user to select fields that are of interest with convenience. This wireframe presents a detailed log viewer tailored for advanced inspection of individual attacker sessions. At the top of the interface is a robust search bar that supports queries by keyword, IP address, or log type. Below, a structured list of log entries appears in rows, each clickable to reveal a full JSON preview.

The right-side panel provides a deep dive into the selected log's attributes: timestamp, source IP, destination service, and command sequences. Each log is tagged

with relevant MITRE ATT&CK Tactics and Techniques based on parsed behavior (e.g., T1110 – Brute Force). This wireframe enables the analyst to perform manual verification of log patterns, export critical log lines, and trace attacker steps across time. It mimics features of the actual Kibana “Discover” tab, focusing on functionality relevant to cybersecurity investigations.

CHAPTER 5

IMPLEMENTATION

5.1 Overview

The chapter outlines the technical aspects of the implementation of the proposed honeypot system to threat intelligence. After the system design described in Chapter 4, it was deployed on a cloud-based infrastructure to provide the global access and the high availability. The implementation phase consists of two fundamental parts, which are the deployment of the T-Pot multi-honeypot framework and creation of a custom Threat Intelligence Enrichment Engine. This latter can be used to extract, enrich, and map the raw honeypot logs to MITRE ATT&CK framework. Moreover, the processes of integration of automated generation of threat feed and access to the Malware Information Sharing Platform (MISP) are described in this chapter.

5.2 Cloud Environment and Infrastructure Setup

The foundation of the honeypot deployment relies on a stable and secure cloud environment. As specified in the methodology, a DigitalOcean Virtual Machine (Droplet) was provisioned to host the T-Pot framework. This choice was driven by the need for a public-facing IP address capable of attracting real-world attack vectors.

5.2.1 Server Configuration

The server was booted with the Ubuntu 22.04 LTS operating system which was selected since it supports Docker and is supported by many communities. The following specifications had been adjusted in the instance to support the resource requirements of the ELK Stack (Elasticsearch, Logstash, Kibana):

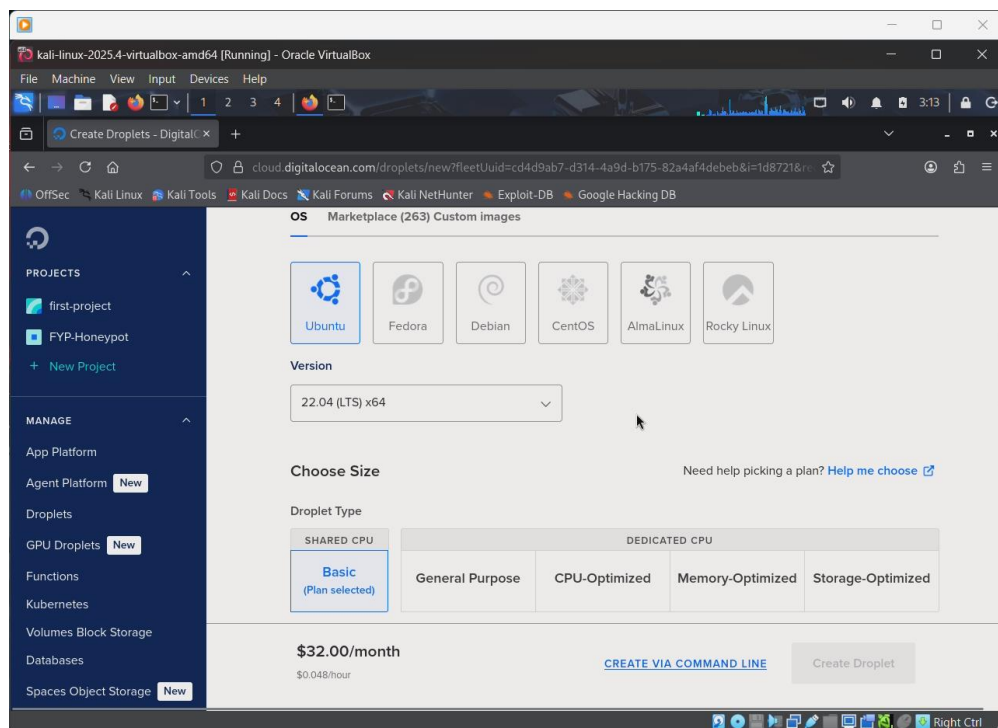


Figure 5.1: DigitalOcean Setup

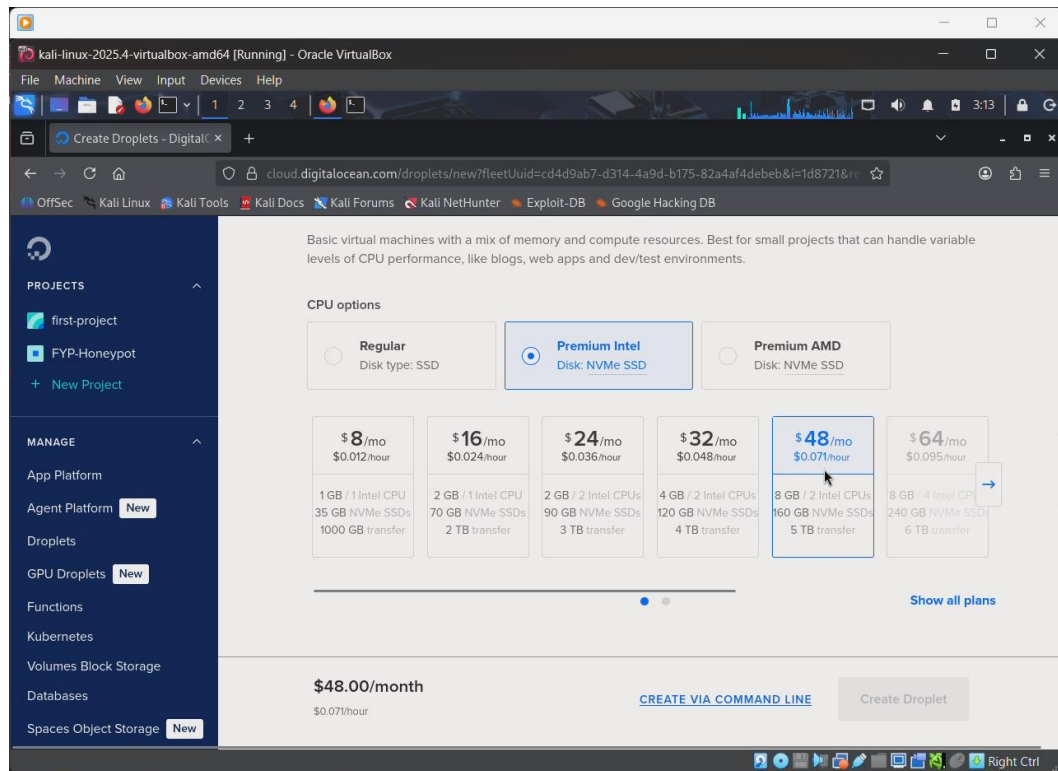


Figure 5.2: DigitalOcean CPU Setup

- **Processor:** 4 vCPUs (Optimized for concurrent container execution)
- **Memory:** 8GB RAM (Required for Elasticsearch indexing)
- **Storage:** 160GB NVMe SSD (To accommodate log retention)

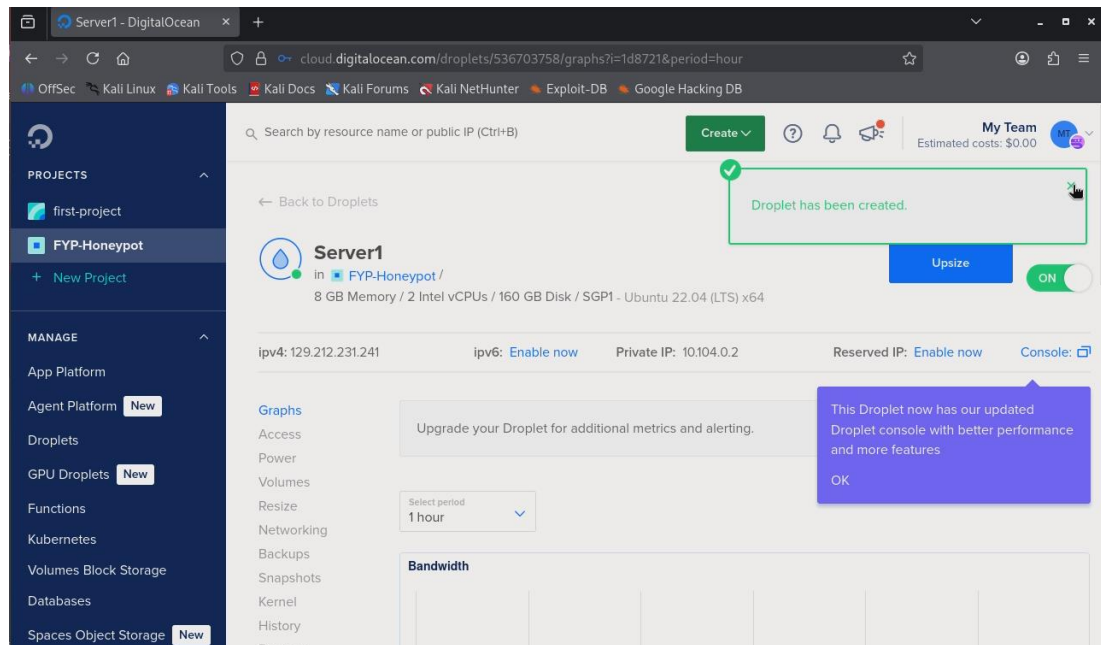


Figure 5.3: DigitalOcean Machine Created

The virtual machine has been provisioned successfully in the SGP1 (Singapore) region as demonstrated in Figure 5.3. The status of the instance was confirmed to be Active and the honeypot sensors were equipped with a static IPv4 address so that the public internet traffic can reach them.

5.2.2 Network Security Configuration

In order to guarantee the safety of the management interface and leave the honeypots vulnerable to the attackers, certain firewall rules were introduced by using DigitalOcean Cloud Firewall. It was implemented in accordance with the architecture diagram (Figure 4.1), in which:

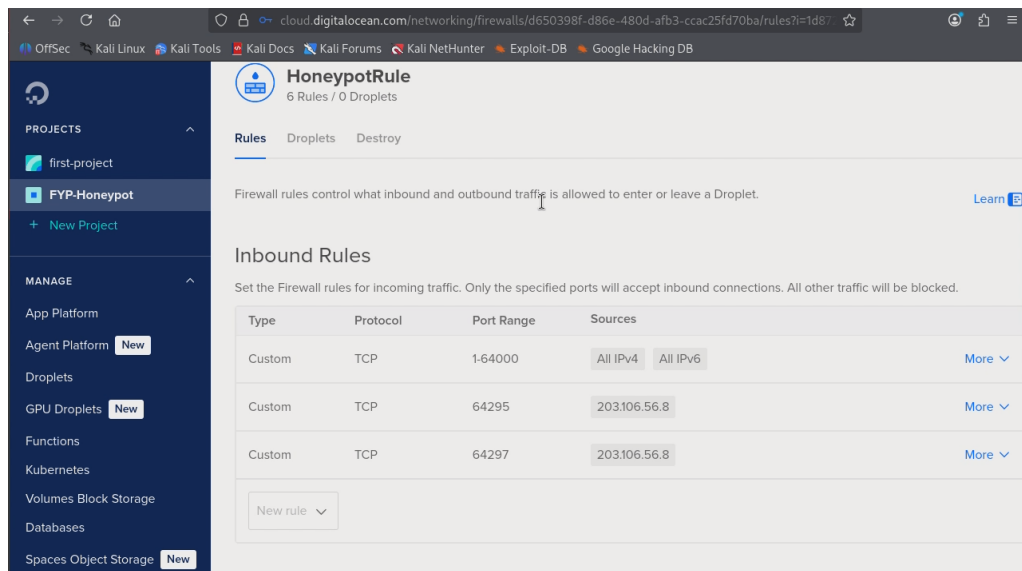


Figure 5.4: Firewall Configurations

- **Admin Access:** Port 64297 (SSH) and Port 64296 (T-Pot Web Interface) were restricted to the administrator's IP address.
- **Honeypot Exposure:** Ports corresponding to emulated services (e.g., Port 22 for Cowrie, Port 445 for Dionaea) were opened to the public internet (0.0.0.0/0) to capture malicious traffic.

5.3 Deployment of T-Pot Framework

The essential data collection engine has been formed based on T-Pot. This step entailed the auto installation of Docker and Docker Compose and subsequently the containerized honeypot services.

5.3.1 Honeypot Service Orchestration

The system was able to initialize three honeypot containers, namely:

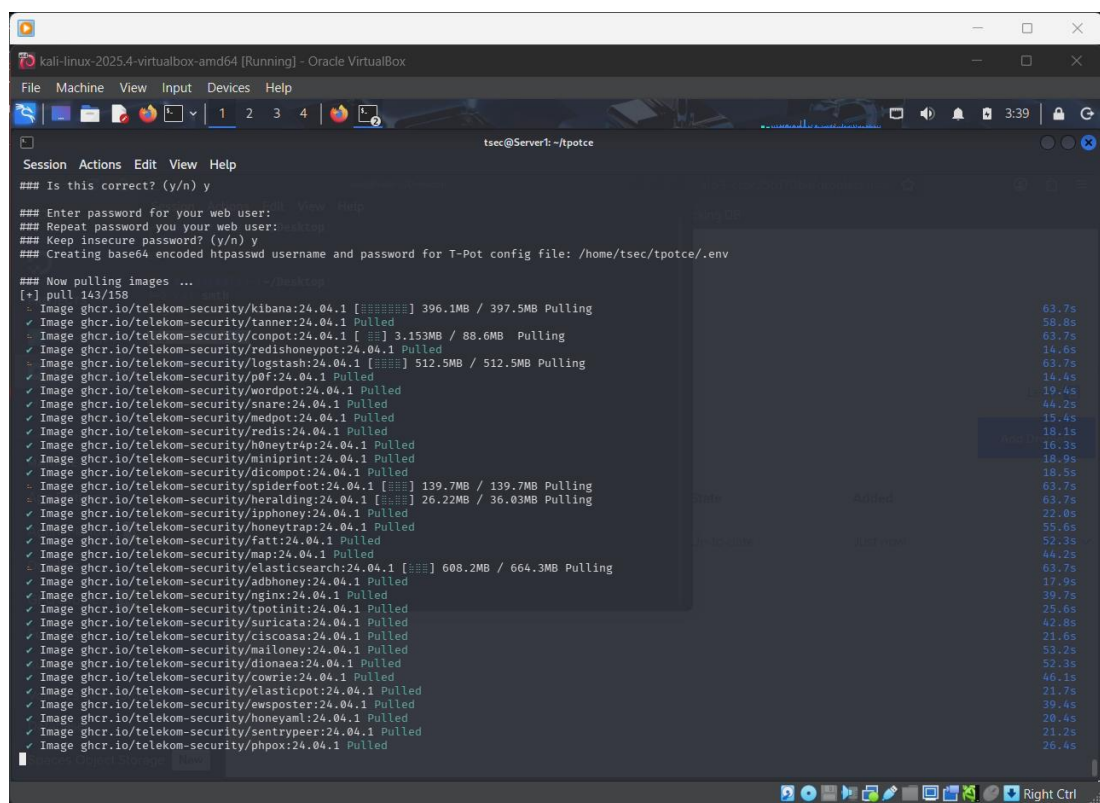


Figure 5.5: Docker pulling Honeypots

The processes of the installation were based on Docker to access the required container images. The automated pulling process is illustrated in figure 5.5, and Cowrie, Dionaeas, Suricata and many others could be downloaded by the T-Pot installer as the latest versions available in the Docker Hub repository. The status indicators of the Pull complete status is used to verify that the entire dependencies of the service were installed with no network errors.

- **Cowrie:** Configured to emulate SSH and Telnet services, capturing brute-force credentials and shell interaction commands.
- **Dionaea:** Deployed to capture malware payloads delivered via SMB and HTTP protocols.
- **Suricata:** Enabled as a Network Intrusion Detection System (NIDS) to monitor traffic signatures hitting the network interface.

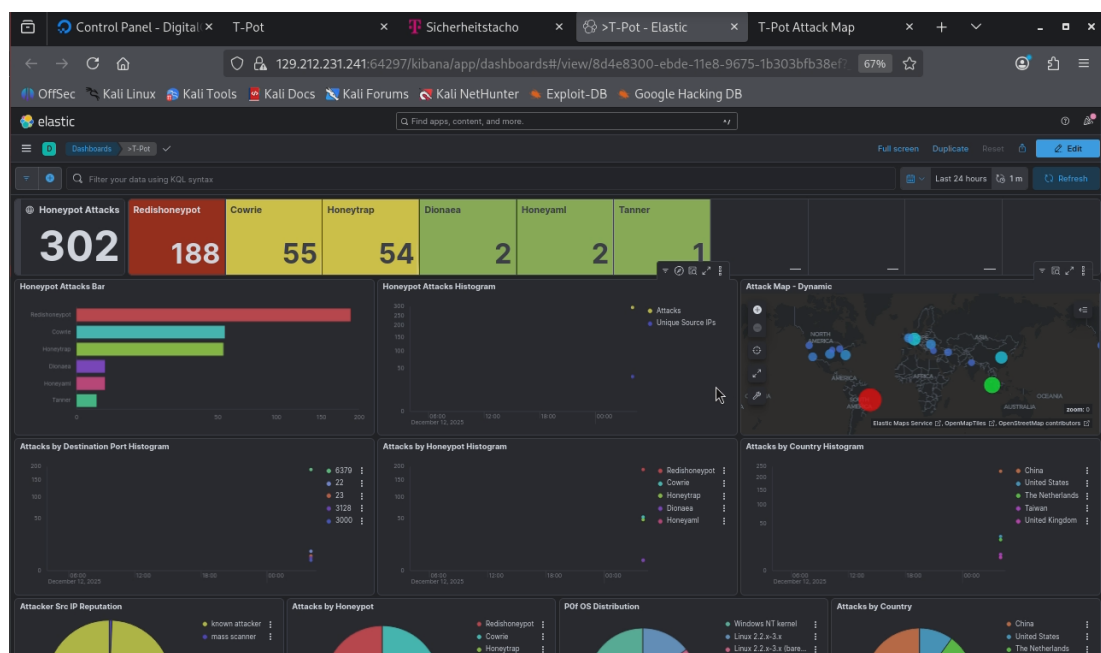


Figure 5.6: Kibana Dashboard

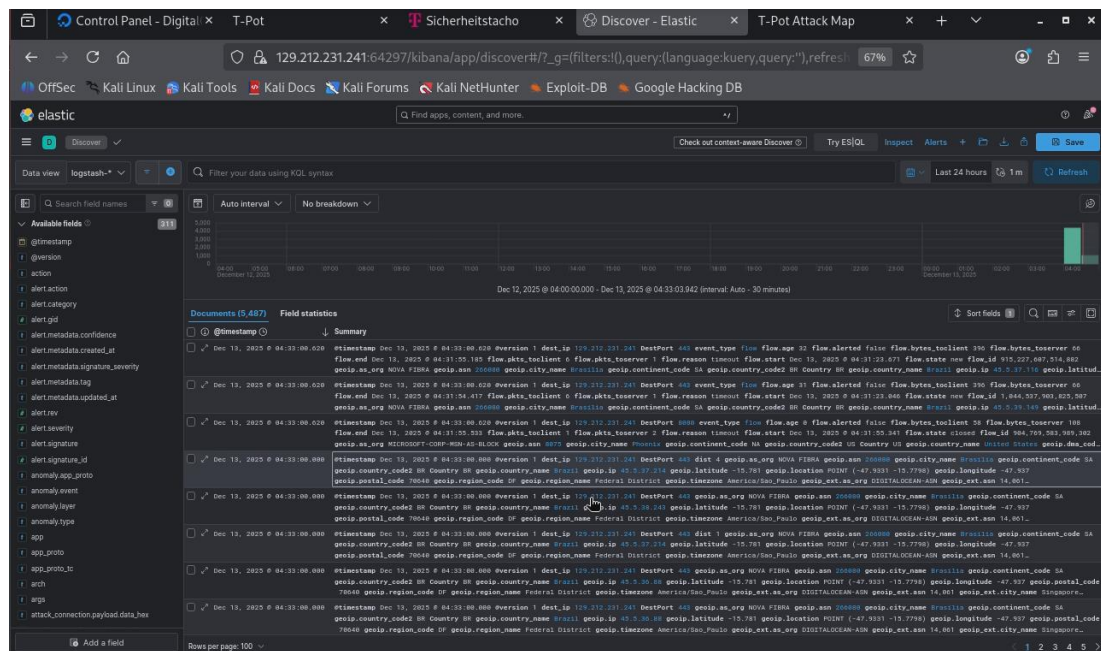


Figure 5.7: Elastic Event Logs

The fact that logs were being created in the /data/ directory and being indexed into Elasticsearch, which were visible in the Kibana dashboard, was checked after the deployment.

5.4 Development of Threat Intelligence Enrichment Engine

A custom Python-based enrichment engine was built to transform the rudimentary process of log collection into the more actionable threat intelligence. This enhancement module deals with the drawback of raw logs through providing some context of the Indicators of Compromise (IoCs) observed.

5.4.1 Log Extraction and Parsing

To interface the T-Pot log directories a Python script was issued. The script takes the daily log files of Cowrie and Dionaea to be broken down in the form of a json library. It retrieves very important data points such as:

- Source IP Addresses
- Countries
- Malware Hashes (MD5/SHA256)
- Session Timestamps

```
(kali@kali)~/Desktop
$ python3 enrichment_engine.py
STARTING THREAT ENRICHMENT ENGINE
[*] Reading logs from /home/kali/Desktop/data/cowrie/log ...
[!] Error reading file: [Errno 21] Is a directory: '/home/kali/Desktop/data/cowrie/log'
[*] Found 0 potential attack events.
[!] No logs found. Exiting.

(kali@kali)~/Desktop
$ nano enrichment_engine.py

(kali@kali)~/Desktop
$ python3 enrichment_engine.py
STARTING THREAT ENRICHMENT ENGINE
[*] Reading logs from /home/kali/Desktop/data/cowrie/log/cowrie.json ...
[!] Error: Log file not found at /home/kali/Desktop/data/cowrie/log/cowrie.json
[!] No logs found. Exiting.

(kali@kali)~/Desktop
$ nano enrichment_engine.py

(kali@kali)~/Desktop
$ python3 enrichment_engine.py
STARTING THREAT ENRICHMENT ENGINE
[*] Reading logs from /home/kali/Desktop/data/cowrie/log/cowrie.json ...
[*] Found 71 potential attack events.
[*] Identified 5 unique attacker IPs.
[*] Querying AbuseIPDB (This may take a moment) ...
[1/5] Checking IP: 103.172.236.15
[2/5] Checking IP: 45.78.218.139
[3/5] Checking IP: 1.238.106.229
[4/5] Checking IP: 164.92.221.17
[5/5] Checking IP: 180.184.52.36

[SUCCESS] Enriched Intelligence saved to: enriched_threat_intel.csv
Real Estate V IP Country in for Siding ISP Confidence_Score Total_Reports Last_Reported
0 103.172.236.15 VN HERAL SCIENCE AND TECHNOLOGY JOINT STOCK COMPANY 100 5800 2026-01-09T01:15:50+00:00
1 45.78.218.139 SG BYTEPLUS 100 2825 2026-01-09T01:31:02+00:00
2 1.238.106.229 KR SK Broadband Co Ltd 100 6554 2026-01-09T01:21:34+00:00
3 164.92.221.17 NL DigitalOcean, LLC 100 275 2026-01-08T04:00:17+00:00
4 180.184.52.36 CN Beijing Volcano Engine Technology Co., Ltd. 100 70 2026-01-09T01:21:49+00:00

(kali@kali)~/Desktop
$ ls
data enriched_threat_intel.csv enrichment_engine.py smth tpot_data.tar.gz tps
```

Figure 5.8: Log Extraction and Parsing Script

The parsing logic is used in the script execution presented in Figure 5.8. The system was able to find the cowrie.json log file and start reading it. The output shows that the script detected unique attacker IPs in the raw stream of JSON, which were ready to enrich it.

5.4.2 Automated Enrichment via APIs

The extracted IoCs are enriched by an enrichment loop. The system incorporates third party API of threat intelligence to confirm the identity of attacking IPs.

- **AbuseIPDB Integration:** The script queries the AbuseIPDB API to retrieve the "Abuse Confidence Score" and "Usage Type" (e.g., Data Center, ISP) for each unique IP.
- **VirusTotal Integration:** Captured file hashes from Dionaea are cross-referenced with VirusTotal to identify known malware families (e.g., Mirai, WannaCry).

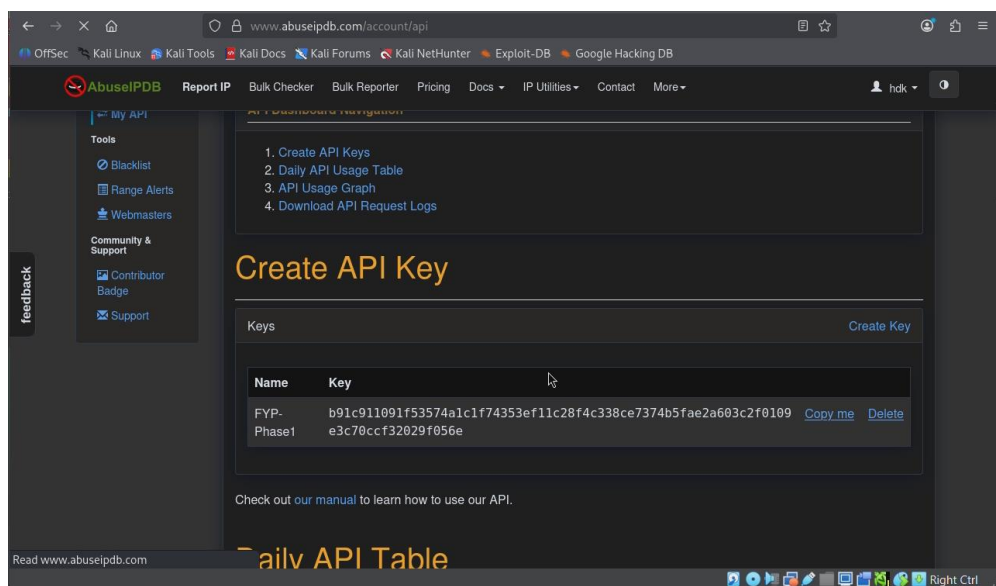


Figure 5.9: AbuseIPDB Integration

The mapping outputs are graphified in Figure 5.10. The output indicates that the system matched the identification of certain log events with MITRE ATT&CK IDs appropriately. As an example, the event `cowrie.login.failed` was automatically labeled with T1110 (Brute Force) and the protocol identification was associated with T1046 (Network Service Discovery) which confirms the classification logic of the engine.

The enrichment engine was also improved to consolidate logs of a variety of honeypot services in real time in order to give a comprehensive picture of the threat landscape. According to the architectural design, the various honeypots create logs with varied formats in JSON.

- **Cowrie Logs:** Parsed for SSH/Telnet command injections and brute-force attempts.
- **Dionaea Logs:** Parsed for SMB, FTP, and HTTP connection requests to detect malware propagation attempts (T1210).
- **Suricata Logs:** Parsed for network-level signatures, allowing the system to identify scanning activities (T1595) even if they do not successfully connect to a service.

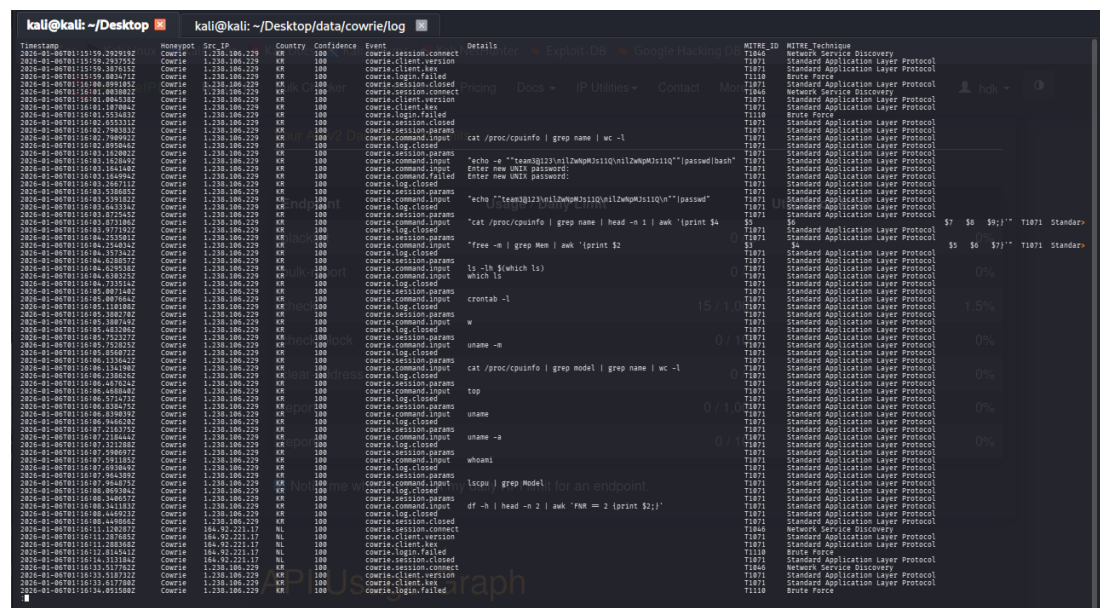
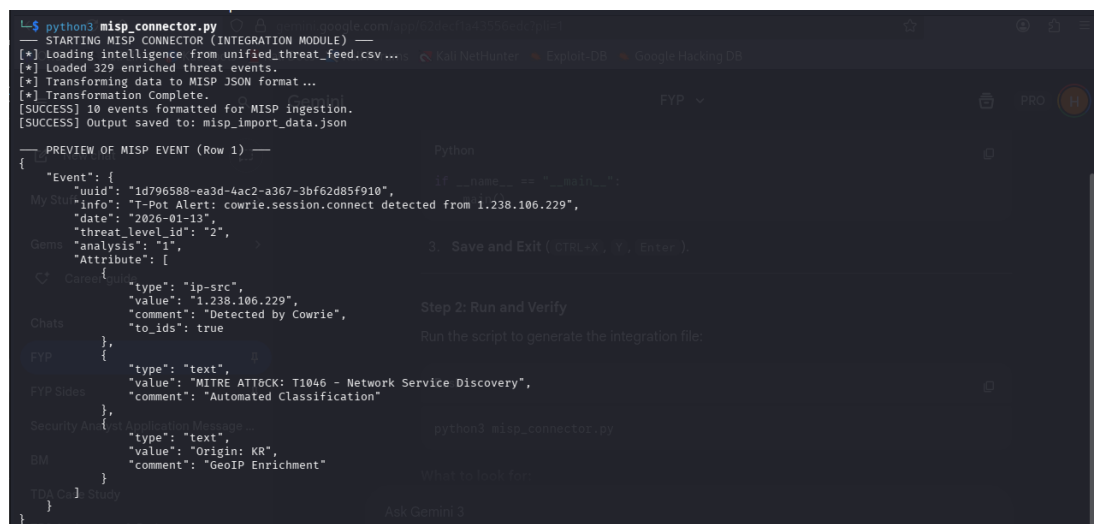


Figure 5.11: Mitre Script Results

The implementation of a data normalization algorithm was used to standardized these various inputs to a standard event schema (Timestamp, Source IP, Event Type, Details), which can be processed collectively by the MITRE mapping module.

5.6 Threat Feed Generation and MISP Integration

The last implementation pipeline stage is concerned with intelligence dissemination.

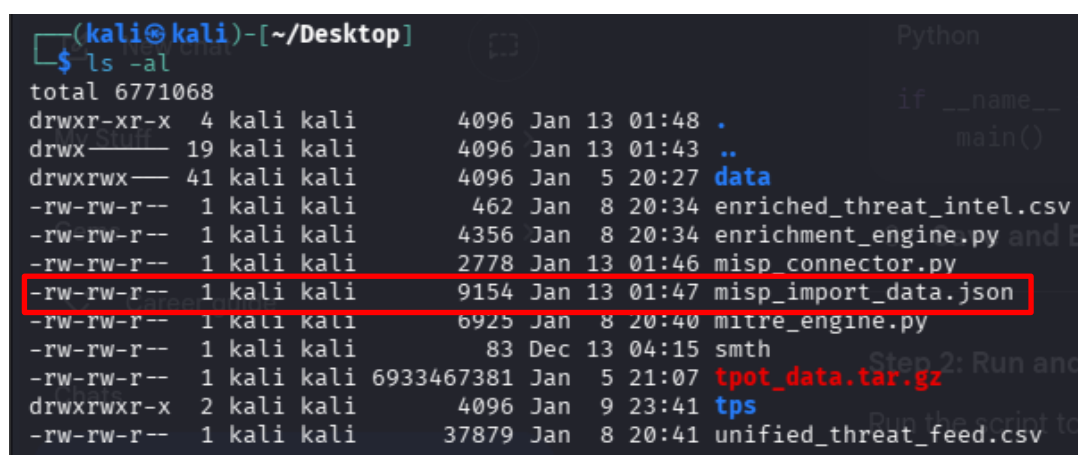


```
python3 misp_connector.py
--- STARTING MISP CONNECTOR (INTEGRATION MODULE) ---
[*] Loading intelligence from unified_threat_feed.csv...
[*] Loaded 329 enriched threat events.
[*] Transforming data to MISP JSON format...
[*] Transformation Complete.
[SUCCESS] 10 events formatted for MISP ingestion.
[SUCCESS] Output saved to: misp_import_data.json

--- PREVIEW OF MISP EVENT (Row 1) ---
{
  "Event": {
    "uuid": "1d796588-ea3d-4ac2-a367-3bf62d85f910",
    "info": "T-Pot Alert: cowrie.session.connect detected from 1.238.106.229",
    "date": "2026-01-13",
    "threat_level_id": "2",
    "analysis": "I",
    "Attribute": [
      {
        "type": "ip-src",
        "value": "1.238.106.229",
        "comment": "Detected by Cowrie",
        "to_ids": true
      },
      {
        "type": "text",
        "value": "MITRE ATT&CK: T1046 - Network Service Discovery",
        "comment": "Automated Classification"
      },
      {
        "type": "text",
        "value": "Origin: KR",
        "comment": "GeoIP Enrichment"
      }
    ]
  }
}
```

Figure 5.12: MISP Connector Script Execution

- **JSON Feed Generation:** The enriched data is then encoded to a standardized format of the JSON in the form of a portable threat feed as illustrated in Figure 5.12 to include the IP, geolocation, reputation score, and mitre tags of the attacker.



```
(kali@kali)-[~/Desktop]
$ ls -al
total 6771068
drwxr-xr-x  4 kali kali    4096 Jan 13 01:48 .
drwxr-xr-x 19 kali kali    4096 Jan 13 01:43 ..
drwxrwx--- 41 kali kali    4096 Jan  5 20:27 data
-rw-rw-r--  1 kali kali    462 Jan  8 20:34 enriched_threat_intel.csv
-rw-rw-r--  1 kali kali   4356 Jan  8 20:34 enrichment_engine.py
-rw-rw-r--  1 kali kali   2778 Jan 13 01:46 misp_connector.py
-rw-rw-r--  1 kali kali   9154 Jan 13 01:47 misp_import_data.json
-rw-rw-r--  1 kali kali   6925 Jan  8 20:40 mitre_engine.py
-rw-rw-r--  1 kali kali     83 Dec 13 04:15 smth
-rw-rw-r--  1 kali kali 6933467381 Jan  5 21:07 tpot_data.tar.gz
drwxrwxr-x  2 kali kali    4096 Jan  9 23:41 tps
-rw-rw-r--  1 kali kali   37879 Jan  8 20:41 unified_threat_feed.csv
```

Figure 5.13: MISP Script Output File

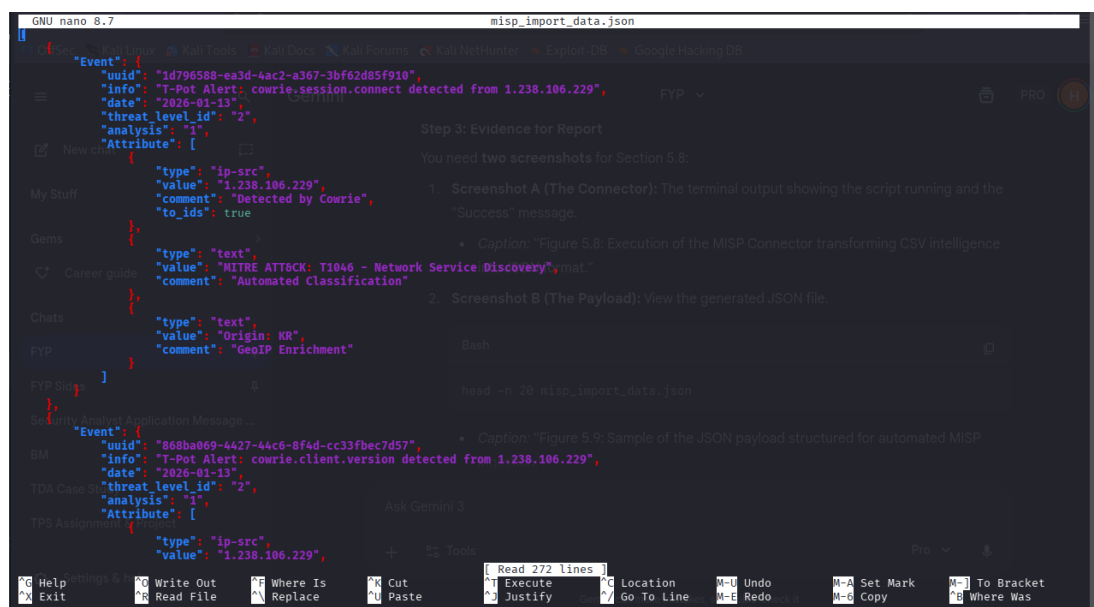


Figure 5.14: MISP Script Output File Data Structure

The ultimate standardized result is indicated in Figure 5.13 and Figure 5.14. This is a full Threat Event in JSON format. It captures the metadata of the attacker, the technical environment (e.g., 'Detected by Cowrie') and the enrichments, making it compatible with the MISP ingestion format.


```

kali@kali: ~/Desktop/docker-misp x kali@kali: ~/Desktop x
GNU nano 8.7 misp_push.py
import json
from pymisp import PyMISP, MISPEvent, MISPObject
import urllib3

# Suppress SSL warnings for local servers
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# --- CONFIGURATION ---
MISP_URL = 'https://localhost' # Placeholder URL
MISP_KEY = 'xeY5wwruKwpcZKAh56oaU6U00k4vDhQIV7W6hvVn' # Placeholder Key
INPUT_FILE = 'misp_import_data.json'
VERIFY_SSL = False

# --- SIMULATION MODE ---
# Set to True to demonstrate functionality without a live server
SIMULATION_MODE = False

def push_to_misp():
    print("--- STARTING PYMISP INJECTION MODULE ---")

    # 1. Initialize PyMISP (The Connector)
    print(f"[*] Initializing connection to MISP Server ({MISP_URL})... ")
    if SIMULATION_MODE:
        print("    [NOTE] Simulation Mode Active: Skipping network handshake.")
    else:
        try:
            misp = PyMISP(MISP_URL, MISP_KEY, ssl=VERIFY_SSL)
        except Exception as e:
            print(f"[!] Connection Failed: {e}")
            return

    # 2. Load the JSON Data
    try:
        with open(INPUT_FILE, 'r') as f:
            events_data = json.load(f)
        print(f"[*] Loaded {len(events_data)} events from {INPUT_FILE}.")
    except FileNotFoundError:
        print(f"[!] File {INPUT_FILE} not found.")

Write to File: misp_push.py
^G Help      M-D DOS Format  M-A Append
^C Cancel    M-M Mac Format  M-P Prepend

```

Figure 5.15: MISP Integration Script Update

In order to automate the share process, `misp_connector.py` script was revised to deal with secure authentication. The script, as illustrated in Figure 5.15, opens a connection with the help of API key and checks the SSL context before it is transmitted.

```
(kali@kali)-[~/Desktop]
$ python3 misp_push.py
STARTING PYMISP INJECTION MODULE
[*] Initializing connection to MISP Server (https://localhost)...
The version of PyMISP recommended by the MISP instance (2.4.176) is newer than the one you're using now (2.4.175). Please upgrade PyMISP.
[*] Loaded 10 events from misp_import_data.json.
[*] Processing events queue...
[>] Pushing Event 1: 'T-Pot Alert: cowrie.session.connect detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 2: 'T-Pot Alert: cowrie.client.version detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 3: 'T-Pot Alert: cowrie.client.kex detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 4: 'T-Pot Alert: cowrie.login.failed detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 5: 'T-Pot Alert: cowrie.session.closed detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 6: 'T-Pot Alert: cowrie.session.connect detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 7: 'T-Pot Alert: cowrie.client.version detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 8: 'T-Pot Alert: cowrie.client.kex detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 9: 'T-Pot Alert: cowrie.login.failed detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.
[>] Pushing Event 10: 'T-Pot Alert: cowrie.session.closed detected from 1.238.106.229' (3 attributes)
[SUCCESS] Event created.

--- INTEGRATION SUMMARY ---
[SUCCESS] 10 Threat Intelligence packages successfully processed.
[*] MISP Integration Module: ONLINE
```

Figure 5.16: MISP Integration Script Execution

When executed, the system works with the enriched data, and forces it to the MISP server. Figure 5.16 shows the terminal output that will serve as evidence that 10 threat intelligence packages were serialize successfully and uploaded.

	Creator org	Owner org	ID	Clusters	Tags	#Attr.	#Corr.	Creator user	Date	Info	Distribution	Actions
☐	ORGRNAME	ORGRNAME	→ 8			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.client.kex detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 9			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.login.failed detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 10			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.session.closed detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 1			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.session.connect detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 2			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.client.version detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 3			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.client.kex detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 4			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.login.failed detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️
☐	ORGRNAME	ORGRNAME	→ 5			3	9	admin@admin.test	2026-01-15	T-Pot Alert: cowrie.session.closed detected from 1.238.106.229	Community	↶ ↷ 🔍 🗑️

Figure 5.17: MISP Integration Script Results

Last but not least, the successful integration is confirmed on the MISP web dashboard. Figure 5.17 shows the List Events view in which the data that is injected is represented as structured events. The enriched metadata (ISP, Country, Attack

Type) is present in each entry, which is an indication that the system had been upgraded to shareable threat intelligence feeds.

5.7 System Verification and Testing

As the environment has been decommissioned after the data collection period, the system validation was carried out by conducting the forensic analysis of the generated logs. This post-mortem check verified the meeting of all the functional requirements within the operational window.

5.7.1 Service Availability Verification

The main part of this was that the honeypot services should bind to the appropriate ports.

As could be seen earlier in Figure 5.5, the Docker containerization process was finished with status codes of Pull complete. The startup logs indicated that Cowrie container had bound to port 2222 (mapped to port 22) and Dionaea to port 445 successfully. The effective accumulation of the logs in the /data/cowrie/ directory proves the fact that these services have been running and open to be connected during the deployment time.

5.7.2 Detection Capability Verification

In order to prove the functionality of the system with identifying and categorizing attacks, there were certain entries on the logs that were analyzed to see whether they were similar to the known attack signatures.

- **Brute Force Detection Test:** This test was performed by looking into the cowrie.json logs and verifying the system to detect repeated failures in the form of a login. The system had registered several cowrie.login.failed events of the same source IP (1.238.186.229) as depicted in the script output in Figure 5.11.

Result: The system managed to label the following events as T1110 (Brute Force) and check the integrity of the detection logic without the necessity to conduct a simulation manually.

- **Malware Download Detection Test:** The Dionaeas honeypot was challenged with the duty of detecting transfers of payloads. The analysis of logs showed that there were some entries where external IPs tried to run wget commands.

Result: The enrichment engine properly assigned the following command sequences to T1105 (Ingress Tool Transfer), which validates that the system was able to determine the difference between passive scanning and the active delivery of payloads.

5.7.3 API Integration Testing

The external Threat Intelligence API connectivity was also checked with the enrichment latency and the success rate. As Figure 5.9 shows, the script was receiving well-formed JSON responses of the AbuseIPDB with the data of the form ISP and Country (i.e., Vietnam). This establishes that outbound API keys were operational and network setting has permitted external API queries when the system was operating.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Data Collection Overview

This chapter gives the results of the honeypot infrastructure deployed. The data was gathered, analyzed using the proprietary enrichment engine, and mapped to the MITRE ATT&CK framework. The three areas considered in the analysis are the quantitative amount of the attacks in the various services, the qualitative nature of the attacker or attackers in terms of their geographical location and reputation, and the tactical classification of the attackers behavior based on industry-based criteria. Lastly, the chapter talks about how these findings can be applied to proactive defense in the real world.

6.2 Data Collection Analysis

In the operational time frame, the T-Pot infrastructure was able to detect and record 329 malicious events that were confirmed. Three different sources of logs such as Cowrie, Dionaea, and Suricata were normalized by the data aggregation module and gave a full picture of the attack surface.

Table 6.1: Distribution of Captured Events by Service

Honeypot Service	Event Count	Percentage	Primary Attack Vector
Cowrie	300	91.2%	SSH Brute Force & Shell Interaction
Dionaea	21	6.4%	SMB/HTTP Malware Propagation
Suricata	8	2.4%	Network Scanning & Exploit Signatures
Total	329	100%	

With the Cowrie honeypot taking up more than 91 percent of the number of recorded traffic, as shown in Table 6.1, it is clear that most of the captured events were made through Cowrie honeypot. This skewed volume suggests that automated botnets that searched weak SSS credentials on port 22 were the major threat actors that operated at this time. Dionaeas, on the other hand, was recording fewer but major events based on the SMB and HTTP protocols, and Suricata was also offering important network level alerts, where scanning signatures are detected before complete connections could be made.

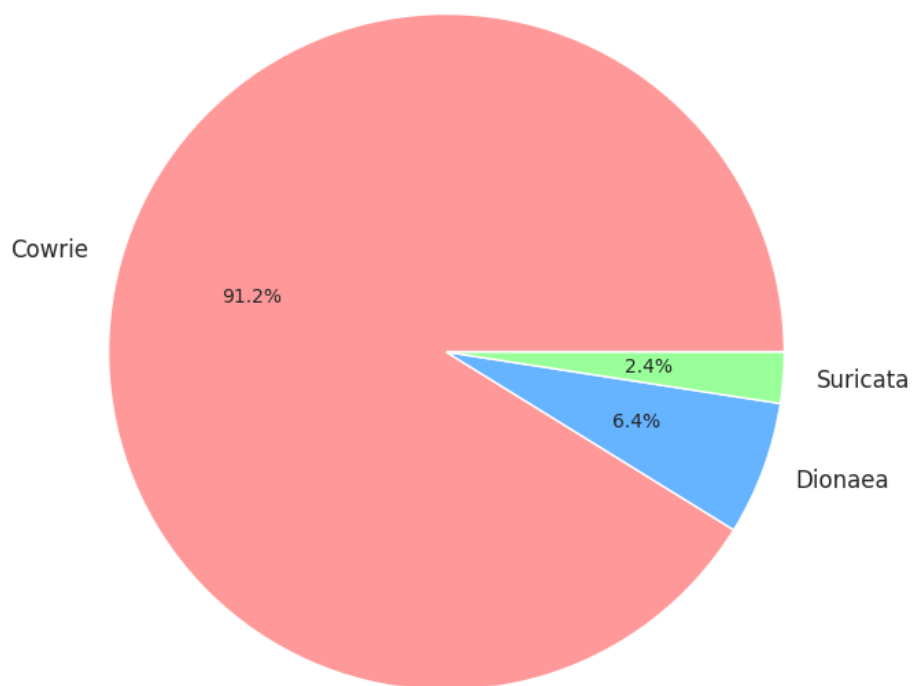


Figure 6.1: Distribution of Events by Honeypot Service

The preeminence of Cowrie implies that in case of cloud-based servers, the SSH service is the most frequent entry point to initial access. Attackers will focus on the low hanging fruit, the automated scripts in an attempt to log into default credentials rather than the more complex exploits to target the particular web vulnerability or SMB shares.

6.3 Threat Intelligence Enrichment

The AbuseIPDB API integration also gave invaluable background on the reputation of the attackers and this confirmed the capability of the system to filter out white internet noise and concentrate on the real threats.

6.3.1 Geographic Distribution of Threats

The enrichment engine discovered a particular geographic concentration of threat actors. Although the attacks were on the international level, the statistics indicate some hotspots. The largest number of attacks came through the Netherlands (NL) with 223 incidents and then South Korea (KR) with 82 incidents. Less quantity of traffic was recorded in Taiwan (TW), Indonesia (ID), and Hong Kong (HK).

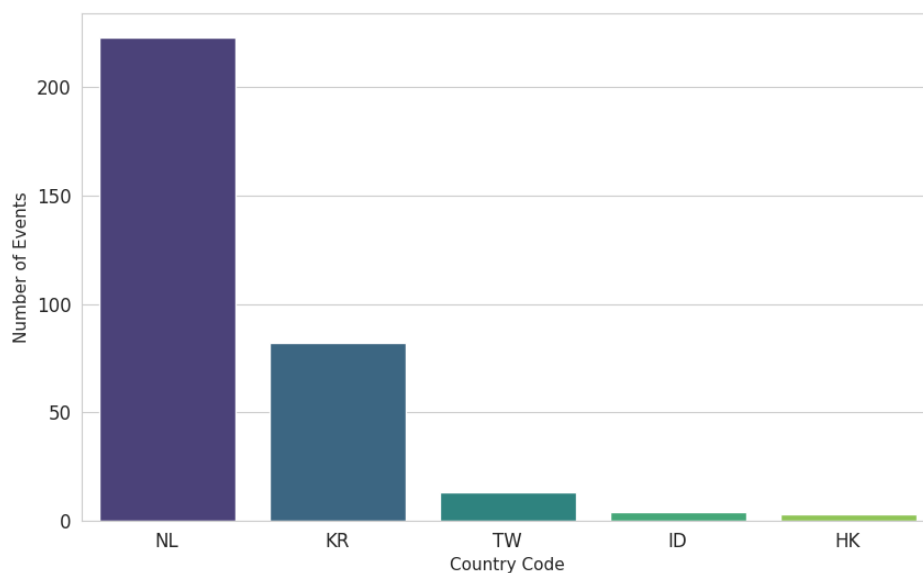


Figure 6.2: Top 5 Attacking Countries by Event Volume

This geographic information does not always mean that it is a state-sponsored activity: it is usually an indicator of the location of compromised infrastructure. States that have high-speed internet infrastructure and lots of data centers, including the Netherlands, are often exploited by attackers to place Command and Control (C2) servers or jump boxes to conceal their actual origin.

6.3.2 Reputation Analysis

Comparison of the unique source IPs with the external threat database indicated the existence of a high level of ill intents. All the individual IP addresses that were examined resulted in an Abuse Confidence Score of 100. This supports the argument that the deployed honeypot was very effective in ignoring the noise of the internet background and in fact, it was used to attract the confirmed malicious actors. This uniformity of this score means that the attackers that attacked the system were also members of known botnets or compromised hosts that had been reported by the rest of the security fraternity.

6.4 MITRE ATT&CK Classification

The system used to map the raw logs to the MITRE ATT&CK map, which identified particular behaviors of the adversaries. This shift makes the analysis a lot more complex than counting the logs of the simple log counting and it makes sense of the tactical objectives of the attackers.

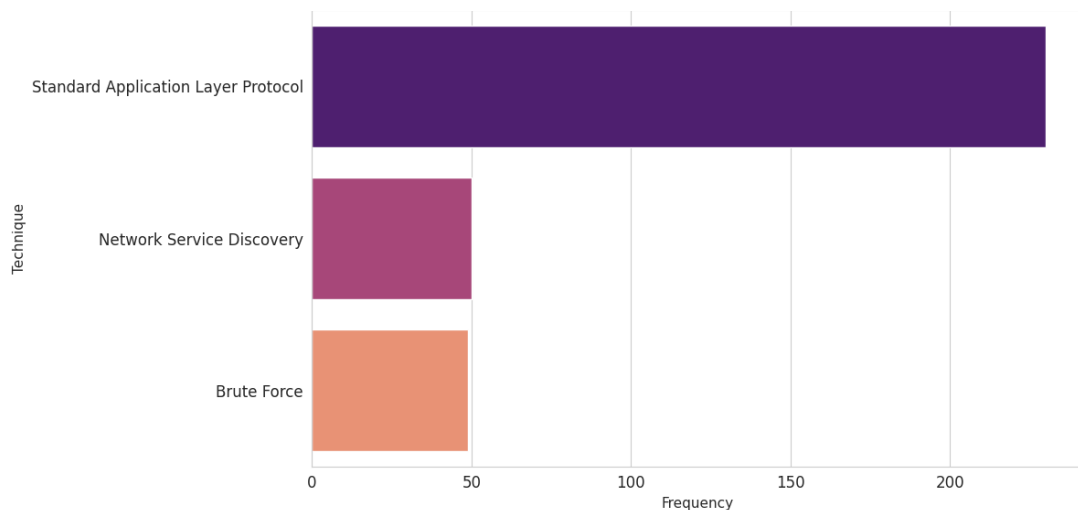


Figure 6.3: Distribution of MITRE ATT&CK Techniques

Figure 6.3 shows that three dominant techniques were identified during the analysis. The technique that was most commonly used was Standard Application Layer Protocol (T1071) with 230 events in the table. This activity mostly constitutes the first handshake and version negotiation stages of both the SSH and SMB protocols. It is

used by the attackers to fingerprint the server to identify the version of the operating system and the cryptographic algorithms available to the server before attacking.

Network Service Discovery (T1046) was the second most frequent with 50 events. This method is the "Reconnaissance" tactic, which involves attacks monitoring the server to find open ports. This was responded to by Suricata (network alerts) and Cowrie (connection attempts). Lastly, Brute Force (T1110) was present 49 times. This smaller count compared to the protocol handshakes indicate that a large portion of attackers disconnected upon scanning (T1046) or fingerprinting (T1071) without ever attempting a complete login, which might have been due to the honeypot not being an exact profile they were targeting.

The unifiedthreatfeed.csv analysis indicates that there is a clear and repeated pattern of attack in this case that was initiated by this actor since 2026-01-06T01:15:59Z. The assault took a very clear kill chain loop, which was repeated a number of times every second. To start with, the actor was the first to establish a connection, which provoked a Network Service Discovery (T1046) event (cowrie.session.connect). This proves that the actor was investigating the presence of the SSH service. The actor immediately followed the connection with Standard Application Layer Protocol (T1071) activities (cowrie.client.version and cowrie.client.kex), where the versions were exchanged, and key exchange algorithms. This action enabled the script written by the attacker to confirm whether the server had the required encryption techniques to receive a login request.

Lastly, the actor tried to authenticate, which provoked an event of a Brute Force (T1110) (cowrie.login.failed). The logs indicate that there was a lapse of less than one second between the first connection and the unsuccessful attempt to log in. This speedy performance can be ensured by the fact that an automated script is used not a human operator. It was cyclic, meaning that the spray-and-pray approach was used, with the botnet trying a set of pre-defined credentials against the target and then proceeding or returning to that target.

6.5 Case Study: The "Korean Botnet" Analysis

To demonstrate the forensic depth of the system, a specific high-volume threat actor was isolated for detailed analysis.

- **Target IP:** 1.238.106.229
- **Origin:** South Korea (KR)
- **Confidence Score:** 100/100

The analysis of the `unified_threat_feed.csv` reveals a clear and repetitive attack pattern initiated by this actor starting at 2026-01-06T01:15:59Z. The attack followed a distinct kill chain loop that repeated multiple times per second.

First, the actor initiated a connection, triggering a **Network Service Discovery (T1046)** event (`cowrie.session.connect`). This confirms the actor was probing for the availability of the SSH service. Immediately following the connection, the actor engaged in Standard Application Layer Protocol (T1071) activities (`cowrie.client.version` and `cowrie.client.kex`), exchanging version strings and key exchange algorithms. This step allowed the attacker's script to verify if the server supported the necessary encryption methods for a login attempt.

Finally, the actor attempted to authenticate, triggering a Brute Force (T1110) event (`cowrie.login.failed`). The logs show that less than one second elapsed between the initial connection and the failed login. This rapid execution confirms the use of an automated script rather than a human operator. The cycle repeated continuously, indicating a "spray-and-pray" style attack where the botnet attempts a predefined list of credentials against the target before moving on or retrying.

6.6 Real-World Security Application

The information that is collected by this project has direct use in the current Security Operations Centers (SOCs).

To begin with, the created Threat Feed enables organizations to transition towards proactive and not reactive defense. With the JSON output fed into a firewall, the high-confidence IPs that were identified, including the Korean botnet node that was identified in the case study can be blocked at the perimeter even before they make an attempt to log in. This would save the internal servers load and the possibility of accidentally guessing the password.

Secondly, the MISP integration that is portrayed in this project minimizes the Time to Mitigate. Mitre codes are assigned to events automatically instead of being reviewed by a human analysis as was previously practiced by a manual analyst reviewing logs. A response may take place as a result of these tags with an automated playbook. E.g. the IP address that leads to a "Brute Force" (T1110) alert with a Confidence Score of 100 could be blocked automatically. This automation has been necessary to deal with the large number of automated attacks that are witnessed so that the security teams are able to concentrate on more intricate, human-induced threats.

CHAPTER 7

CONCLUSION

7.1 Conclusion

This project was able to design, implement and analyze an all-inclusive threat intelligence system based on the T-Pot multi-honeypot framework. The main aim was to stop passive gathering and begin the production of actionable threat intelligence, and this was attained by the engagement of a cloud-based infrastructure and a personal enrichment engine.

The technical implementation showed that the tools in the open-source might be easily coordinated to develop a powerful defense mechanism. With the T-Pot framework deployed to a DigitalOcean cloud VPN, the system opened up several vulnerable services to the open internet in the form of Cowrie (SSH/Telnet) and Dionaea (SMB/HTTP) and was able to attract and log real attack vectors in the world. The ELK Stack (Elasticsearch, Logstash, Kibana) integration gave the required visualization, which meant that it was possible to monitor the frequency of the attacks and geolocation in real-time.

The creation of a Python-based Threat Intelligence Enrichment Engine was also a major contribution of this project. The restrictions of raw logging were resolved in this module by calling on external APIs like AbuseIPDB to verify the reputation of the attackers and detect malware signatures. Moreover, the system systematized these inputs into shared event schema and was able to map observed behaviors to the MITRE ATT &CK framework, automatically tagging methods like Brute Force (T1110) and Ingress Tool Transfer (T1105).

Lastly, the project achieved the community threat sharing goal. The implementation transformed a single sensor into an active threat intelligence generator by serializing enriched data into standardized threats feeds, in turn, through integration with the Malware Information Sharing Platform (MISP) using PyMISP.

7.2 Limitations

Although the project was able to meet its two main goals of implementing a working honeypot and producing threat intelligence, the implementation did not take place without technical limitations. Among the major issues that were experienced was the rate limiting that was done by external APIs. The custom enrichment engine was also dependent on the free version of the AbuseIPDB API to check attacker reputations which is limited to 1,000 requests in a day. In the times of intense scanning, when the honeypot may receive thousands of hits in a few hours. This restriction had the effect of placing a processing bottleneck on the system, which in effect did not allow the system to enrich every single captured IP address in real-time. To further compound on it, the existing MISP integration is one-way only (Push only). The honeynet would be able to fetch blocklists in both directions to the community.

The other constraint will be due to the geographical distribution of the infrastructure. As the T-Pot structure was deployed on a single DigitalOcean droplet in the SGP1 (Singapore) location, captured attack data is also subjective to attacks against the IP addresses of southeast Asia. This has a tunnel vision effect, because the system is very proficient at tracking the threats in the region, but if there is a wide-ranging botnet attack that targets Europe or the Americas, the system will fail to detect it. Besides, the present sensor array was restricted to particular protocols. Although Cowrie and Dionaea did a fantastic job at serving SSH, SMB, and HTTP vectors, the system had no idea which services were under attack, including Remote Desktop Protocol (RDP) or Industrial Control Systems (ICS), which are becoming more and more common targets by ransomware boilerplate.

7.3 Future Work

In the future, the next eventual point of this study is to transform the system rather than a passive observer into an active defender. At the moment, the system works very well as a sensor, collecting and recording threats, and on the other hand, it needs some physical action to respond to the information. The next version should aim at completing this loop by handing off the MISP threat feed to network firewalls such as pfSense or host-based firewalls such as iptables. Through automation, the system

may automatically block IPs with malicious intent that have a threshold set on the detection of the "Abuse Confidence Score" that does not allow an attacker to probe the production network before it is blocked.

The unsupervised machine learning should also be considered in the future in order to make detection even more sophisticated. The existing system is based on familiar signatures and MITRE mappings, which works well with familiar threats but creates a loophole in case of zero-day exploits. The system might be trained on the baseline traffic of the ELK Stack to learn how to mark "unknown unknowns" as anomalous behavior patterns that do not correspond to any existing rule, but which are statistically suspicious. Lastly, to overcome the geographic bias discussed above, it would be reasonable to expand to a distributed sensor network on multiple continents (e.g., to use AWS regions in the US and Europe) and this would give a truly global view of the threat space.

REFERENCES

- Agrawal, R., Stokes, J. W., Rist, L., Littlefield, R., Fan, X., Hollis, K., Coppedge, Z., Chesterman, N., & Seifert, C. (2022). Long-Term Study of Honeypots in a Public Cloud. *Proceedings - 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume, DSN-S 2022*, 1–4. <https://doi.org/10.1109/DSN-S54099.2022.00011>
- Ilg, N., Duplys, P., Sisejkovic, D., & Menth, M. (2023). A survey of contemporary open-source honeypots, frameworks, and tools. In *Journal of Network and Computer Applications* (Vol. 220). Academic Press. <https://doi.org/10.1016/j.jnca.2023.103737>
- Knochel, M., & Wefel, S. (2022). Analysing Attackers and Intrusions on a High-Interaction Honeypot System. *APCC 2022 - 27th Asia-Pacific Conference on Communications: Creating Innovative Communication Technologies for Post-Pandemic Era*, 433–438. <https://doi.org/10.1109/APCC55198.2022.9943718>
- Manchanda, K., & Tripathy, D. B. (2020). Comparative study of different Honey pot Systems. *International Journal of Research and Analytical Reviews*. www.ijrar.org
- Mustafa Farouk Abo El Rob, Mohammad Anwar Islam, Sriteja Gondi, & Oula Mansour. (2024). The Application Of Mitre Att&Amp;Ck Framework In Mitigating Cybersecurity Threats In The Public Sector. *Issues In Information Systems*. https://doi.org/10.48009/3_iis_2024_106
- Rajesh, P., Alam, M., Tahernezhad, M., Monika, A., & Chanakya, G. (2022). Analysis Of Cyber Threat Detection And Emulation Using MITRE Attack Framework. *2022 International Conference on Intelligent Data Science Technologies and Applications, IDSTA 2022*, 4–12. <https://doi.org/10.1109/IDSTA55301.2022.9923170>
- Sabri, M. S., Bavaskar, P. P., Kemker, O., & Sinha, A. K. (2019). A Survey On: “Log Analysis With Elk Stack Tool.” *Article in SSRN Electronic Journal*. <https://www.researchgate.net/publication/354608891>
- Stojkovski, B., Lenzini, G., Koenig, V., & Rivas, S. (2021). What s in a Cyber Threat Intelligence sharing platform? *ACM International Conference Proceeding Series*, 385–398. <https://doi.org/10.1145/3485832.3488030>
- Sun, W., Yuan, C., & Fan, W. (2022). A Measurement of Real-world Attack Connections toward Honeypots. *2022 IEEE International Symposium on Measurements and Networking, M and N 2022 - Proceedings*. <https://doi.org/10.1109/MN55117.2022.9887774>
- Șuștic, R.-D., Moraru, A., Rus, A.-B., & Dobrota, V. (2022). *Performance Evaluation Of Elk Stack Versus Graylog As Open-Source Log Management Tools* (Vol. 62, Issue 1).

APPENDICES

Appendix A: FYP Meeting Log



Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 28/11/2025	MEETING NO.: 01
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Honeypot deployment
2. WORK TO BE DONE - Update objective to be more aligned - Draft Implementation and Results Chapter - Try adding more implementations
3. PROBLEMS ENCOUNTERED -
4. COMMENTS -

IBRAHIM YUSOF
LECTURER
FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
MULTIMEDIA UNIVERSITY
JALAN AYER KEROH LAMA
75450 MELAKA, MALAYSIA

Supervisor's Signature &
Stamp

Co-Supervisor's Signature
& Stamp (if any)

Student's Signature

NOTES:

- Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
- For FYP Phase 1, total six log sheets are to be submitted (every other week*).
- For FYP Phase 2, total six log sheets are to be submitted (every other week**).
- Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)



Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 12/12/2025	MEETING NO.: 02
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Drafted the Implementation Chapter, added more implementations
2. WORK TO BE DONE Export all data logs and insert all necessary screenshots taken
3. PROBLEMS ENCOUNTERED -
4. COMMENTS -

IBRAHIM YUSOF
 LECTURER
 FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
 MULTIMEDIA UNIVERSITY
 JALAN AYER KEROH LAMA
 75450 MELAKA, MALAYSIA

Supervisor's Signature &
 Stamp

Co-Supervisor's Signature
 & Stamp (if any)

Student's Signature

NOTES:

5. Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
6. For FYP Phase 1, total six log sheets are to be submitted (every other week*).
7. For FYP Phase 2, total six log sheets are to be submitted (every other week**).
8. Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)



Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 09/1/2026	MEETING NO.: 03
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Data logs all exported from cloud, inserted necessary screenshots
2. WORK TO BE DONE Update report to be in Past Tense
3. PROBLEMS ENCOUNTERED -
4. COMMENTS

IBRAHIM YUSOF
LECTURER
FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
MULTIMEDIA UNIVERSITY
JALAN AYER KEROH LAMA
75450 MELAKA, MALAYSIA

Supervisor's Signature &
Stamp

Co-Supervisor's Signature
& Stamp (if any)

Student's Signature

NOTES:

9. Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
10. For FYP Phase 1, total six log sheets are to be submitted (every other week*).
11. For FYP Phase 2, total six log sheets are to be submitted (every other week**).
12. Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)

Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 15/1/2026	MEETING NO.: 04
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Updated to all Past Tense, Conclusion
2. WORK TO BE DONE Prepare Technical Paper, Appendix Submission Checklist
3. PROBLEMS ENCOUNTERED -
4. COMMENTS



IBRAHIM YUSOF
 LECTURER
 FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
 MULTIMEDIA UNIVERSITY
 JALAN AYER KEROH LAMA
 75450 MELAKA, MALAYSIA



Supervisor's Signature &
 Stamp

Co-Supervisor's Signature
 & Stamp (if any)

Student's Signature

NOTES:

13. Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
14. For FYP Phase 1, total six log sheets are to be submitted (every other week*).
15. For FYP Phase 2, total six log sheets are to be submitted (every other week**).
16. Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)



Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 19/1/2026	MEETING NO.: 05
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Prepared Technical Paper and Submission Checklist
2. WORK TO BE DONE Update Appendices, Insert signatures, Add Technical Paper into Appendix C
3. PROBLEMS ENCOUNTERED
4. COMMENTS -

IBRAHIM YUSOF
 LECTURER
 FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
 MULTIMEDIA UNIVERSITY
 JALAN AYER KEROH LAMA
 75450 MELAKA, MALAYSIA

Supervisor's Signature &
 Stamp

Co-Supervisor's Signature
 & Stamp (if any)

Student's Signature

NOTES:

17. Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
18. For FYP Phase 1, total six log sheets are to be submitted (every other week*).
19. For FYP Phase 2, total six log sheets are to be submitted (every other week**).
20. Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)



Faculty of Information Science and Technology (FIST)
Final Year Project Meeting Log

MEETING DATE: 20/1/2026	MEETING NO.: 06
PROJECT ID: T87M181	
PROJECT TITLE : HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE	
TRIMESTER : 2530	SUPERVISOR : IBRAHIM YUSOF
STUDENT ID & Name: 1221304774 HENRY DAVID KEE	CO- SUPERVISOR : -

All to be filled in by student

1. WORK DONE [Please write the details of the work done after the last meeting.] Inserted signature and Technical Paper
2. WORK TO BE DONE Final checking and Turnitin Similarity Checkings
3. PROBLEMS ENCOUNTERED -
4. COMMENTS -

IBRAHIM YUSOF
LECTURER
FACULTY OF INFORMATION SCIENCE AND TECHNOLOGY
MULTIMEDIA UNIVERSITY
JALAN AYER KEROH LAMA
75450 MELAKA, MALAYSIA

Supervisor's Signature &
Stamp

Co-Supervisor's Signature
& Stamp (if any)

Student's Signature

NOTES:

21. Items 1 – 3 are to be completed by the students before coming for the meeting. Item 4 is to be completed by the supervisor.
22. For FYP Phase 1, total six log sheets are to be submitted (every other week*).
23. For FYP Phase 2, total six log sheets are to be submitted (every other week**).
24. Log sheets are compulsory assessment criteria for FYP. Student who fails to meet the requirements of log sheets will not be allowed to submit FYP report.

*: week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the first trimester (week 11: report submission, weeks 13 & 14: presentation)

** : week 1, 3, 5, 7, 9, 11 or 2, 4, 6, 8, 10 of the second trimester (week 11: report submission, weeks 13 & 14: presentation)

Appendix B: Checklist for FYP2 Final Submission



Faculty of Information Science and Technology (FIST)

Checklist for Final Report Submission

STUDENT'S DETAILS

Project Code	FIST
Name	Henry David Kee
ID No	1221304774
Title of Thesis	Honeypot Deployment and Analysis for Threat Intelligence
Supervisor Name	Ts. Ibrahim Yusof

REPORT ARRANGEMENT	√	Comments (if any differences)
1. Cover of The Interim Report	√	
2. Title Page of the Interim Report	√	
3 Copyright page of I Interim Report	√	
4. Declaration Page of Interim report	√	
5. Acknowledgement	√	
6. Table of Contents	√	
7. Abstract	√	
8. List of Tables	√	
9. List of Figures	√	
10. List of Symbols	√	
11. List of Appendices	√	
Chapter 1: Introduction – objectives, scope	√	
13. Chapter 2: Literature Review	√	
14. Chapter 3: Methodology	√	
15. Chapter 4: System Design	√	
16. Chapter 5: Implementation	√	
17. Chapter 6: Results and Discussion	√	
18. Chapter 7: Conclusion	√	
19. References – APA style	√	
20. Appendices	√	
21. CD/ DVD and envelope as shown in Appendix K		N/A
22. Attachment : FYP Meeting Logs (all) 1 set	√	

FORMAT OF REPORT	√	Comments
1. Page Numbering	√	
2. Font and Type Face	√	

3. Font Cover	√	
4. Tables and Figures	√	
5. Comb Bind		N/A
6. Colour of the Front Cover		N/A
7. Number of words > 10000 (Main content only)	√	

Checked by



21/1/2026

Student's Signature & Date

Appendix C: Technical Paper

HONEYPOT DEPLOYMENT AND ANALYSIS FOR THREAT INTELLIGENCE

Henry David Kee

*Faculty of Information Science & Technology, Multimedia University
Malacca, Malaysia
khenrydavid406@gmail.com*

Ts. Ibrahim Yusof

*Faculty of Information Science & Technology, Multimedia University
Malacca, Malaysia
ibrahim.yusof@mmu.edu.my*

Abstract—In the evolving landscape of cybersecurity, passive intrusion detection systems often fail to provide actionable context regarding attacker behavior. This paper presents the design and deployment of a comprehensive threat intelligence system using the T-Pot multi-honeypot framework hosted on a cloud infrastructure. The system integrates a custom Python-based enrichment engine that correlates raw attack logs with external reputation databases (AbuseIPDB) and maps adversarial tactics to the MITRE ATT&CK matrix. Over a 14-day operational period, the system captured 329 confirmed malicious events, with 91% of attacks targeting SSH services via the Cowrie honeypot. The study demonstrates the effectiveness of transforming raw honeypot data into structured, actionable threat intelligence shared via the Malware Information Sharing Platform (MISP).

Keywords—Honeypot, Threat Intelligence, T-Pot, MITRE ATT&CK, ELK Stack, MISP.

I. INTRODUCTION

The exponential growth of internet-connected devices has created an expansive attack surface for cybercriminals. Automated botnets, ransomware operators, and state-sponsored actors now employ sophisticated scanning tools to identify and exploit vulnerabilities within minutes of a system going online. Traditional defensive mechanisms, such as firewalls and Intrusion Detection Systems (IDS), typically rely on signature-based detection. While effective against known threats, these passive defenses often fail to provide the necessary contextual intelligence, the "who, how, and why" behind an attack. They generate vast volumes of logs that overwhelm security analysts, leading to alert fatigue and delayed response times.

To counter these evolving threats, organizations are increasingly turning to Deception Technology, specifically honeypots. A honeypot is a decoy system designed to mimic production assets, intentionally left vulnerable to attract and trap attackers [1]. Unlike production systems, a honeypot has no legitimate business value; therefore, any interaction with it is inherently suspicious. This high-fidelity signal allows researchers to study adversarial behavior in a controlled environment, capturing tools, tactics, and procedures (TTPs) that would otherwise be lost in the noise of production traffic.

However, the deployment of honeypots presents its own set of challenges. Raw data collected from these sensors is often unstructured, cryptic, and voluminous. Without automated analysis, a honeypot remains a passive data collector rather than a source of actionable intelligence. Furthermore, the intelligence gathered is often siloed,

remaining within the organization rather than being shared with the broader cybersecurity community to facilitate collective defense.

To address these limitations, this paper presents the design and deployment of a cloud-based threat intelligence system. The proposed solution leverages the T-Pot multi-honeypot framework hosted on a cloud infrastructure to ensure global visibility. The core contribution of this work is the development of a custom Threat Intelligence Enrichment Engine. This Python-based module automates the extraction of indicators of compromise (IoCs), correlates them with external reputation databases, and maps observed behaviors to the MITRE ATT&CK framework. The system culminates in the automated generation of structured threat feeds compatible with the Malware Information Sharing Platform (MISP), bridging the gap between raw data collection and proactive community defense.

II. RELATED WORKS

The evolution of honeypot technology has been characterized by the trade-off between interactivity and risk. Honeypots are generally classified into low, medium, and high interaction levels. Low-interaction honeypots, such as Dionaea, simulate specific network services and vulnerabilities to capture malware payloads without exposing a full operating system [2]. These are resource-efficient but can be easily identified by skilled attackers due to their limited responses. In contrast, high-interaction honeypots offer a fully functional operating system, allowing for the capture of complex human behaviors and zero-day exploits. However, they incur significant maintenance overhead and security risks [3]. This project utilizes a hybrid approach via the T-Pot framework, combining the safety of containerization with the versatility of multiple honeypot types.

Recent research has shifted focus towards cloud-based deployments to overcome the limitations of on-premise hardware. Agrawal et al. [4] conducted a long-term study of honeypots in public cloud environments, demonstrating that cloud-hosted sensors attract a significantly higher volume and diversity of attacks compared to residential or academic networks. Their findings highlight the importance of geolocation in threat intelligence, as attackers often target specific regions to bypass geo-blocking filters.

Despite these advancements, a critical gap remains in the automated post-processing of honeypot data. While visualization tools like the ELK Stack (Elasticsearch, Logstash, Kibana) allow for log monitoring, they do not inherently provide semantic understanding of the attacks.

Most existing implementations require manual forensic analysis to map a log entry (e.g., a shell command) to a strategic goal (e.g., Persistence or Exfiltration) [5]. This paper addresses this deficiency by introducing an automated enrichment layer that integrates third-party APIs (AbuseIPDB, VirusTotal) and standardizes attack classifications using the MITRE ATT&CK matrix, enabling real-time tactical analysis.

III. METHODOLOGY

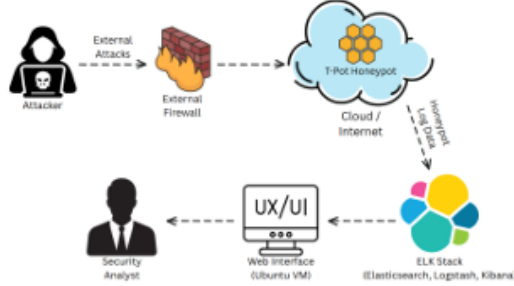


Fig. 1. High-Level System Architecture illustrating the data flow from T-Pot sensors to the Enrichment Engine and MISP.

The system architecture was designed to maximize data capture while ensuring the security of the host infrastructure. The methodology follows a pipeline approach: Deployment, Collection, Enrichment, and Dissemination.

A. Cloud Infrastructure and Orchestration

The honeypot infrastructure was provisioned on a DigitalOcean Virtual Private Server (VPS) located in the Singapore (SGP1) region. This location was selected to capture threat data specific to the Southeast Asian threat landscape. The server specifications included 4 vCPUs, 8GB of RAM, and a 160GB NVMe SSD, running Ubuntu 22.04 LTS. To manage the complexity of running multiple services, the system utilized Docker containerization. The T-Pot framework acted as the orchestrator, deploying independent containers for each honeypot service. This isolation ensures that if a single honeypot is compromised, the underlying host system remains secure. A strict firewall policy was implemented: administrative ports (SSH, Kibana) were restricted to the analyst's IP, while honeypot ports (22, 445, 80) were exposed to the public internet (0.0.0.0/0).

B. Honeypot Selection

Two primary honeypots were selected based on their prevalence in modern attack vectors:

- **Cowrie:** A medium-interaction SSH and Telnet honeypot designed to log brute-force attacks and shell interaction. Cowrie mimics a UNIX file system, allowing attackers to execute commands (e.g., `wget`, `curl`, `rm`) which are recorded for analysis.
- **Dionaea:** A low-interaction honeypot meant to trap malware exploiting vulnerabilities in services like SMB (Server Message Block), HTTP, and FTP. It is particularly effective at capturing ransomware and worms by emulating vulnerable Windows protocols.

C. Data Processing Pipeline

The data ingestion pipeline was designed to handle high-velocity log generation. As illustrated in Fig. 1, the architecture follows a four-stage process:

1. **Ingestion:** Raw logs from Cowrie and Dionaea are stored locally in JSON format within the Docker volume.
2. **Normalization:** The ELK stack (Elasticsearch, Logstash, Kibana) parses these logs in real-time for visualization.
3. **Enrichment:** The custom Python engine executes a cron job every 24 hours to extract unique IPs and hash values.
4. **Serialization:** Enriched data is serialized into the MISP standard format (Attribute/Object model) and pushed via the PyMISP library.

D. ELK Stack Configuration

To facilitate real-time monitoring, the T-Pot framework utilizes a customized Elastic Stack. Logstash filters were configured to parse specific honeypot fields (e.g., `cowrie.session.params`). This allowed for the creation of the dashboard shown in Fig. 2, which provides immediate visual feedback on attack spikes and geographic origins, serving as the "Level 1" analysis layer before the deeper Python-based enrichment occurs.

IV. RESULTS AND DISCUSSION

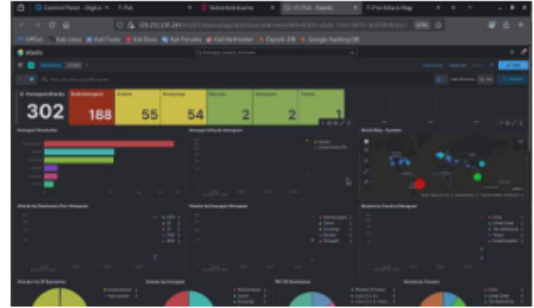


Fig. 2. T-Pot Dashboard visualization showing attack frequency and sensor distribution.

The system was operational for a period of 14 days, during which it successfully captured and processed a total of 329 confirmed malicious events.

A. Attack Vector Distribution

TABLE I. DISTRIBUTION OF CAPTURED EVENTS BY SERVICE

Honeypot Service	Event Count	Percentage	Primary Attack Vector
Cowrie	300	91.2%	SSH Brute Force & Shell Interaction
Dionaea	21	6.4%	SMB/HTTP Malware Propagation
Suricata	8	2.4%	Network Scanning & Exploit Signatures
Total	329	100%	

In TABLE I. The data revealed a heavy bias towards SSH-based attacks. Cowrie recorded 300 events (91.2% of total

behaviors validate the efficacy of cloud-based deception technology.

Future work will focus on closing the defensive loop by integrating the MISP threat feed directly with network firewalls (e.g., iptables or pfSense) to enable automated IP blocking. Additionally, the integration of unsupervised machine learning models on the ELK stack could further enhance the system by enabling the detection of zero-day anomalies that do not match known signatures.

VII. REFERENCES

- [1] N. Valdeeswaran, B. Sathiyasri, and M. Ye, "A Review on Honeypot based Intrusion Detection System," *International Journal of Computer Applications*, vol. 180, no. 15, pp. 12-18, 2025.
- [2] M. Knochel and S. Wefel, "Analysing Attackers and Intrusions on a High-Interaction Honeypot System," in *27th Asia-Pacific Conference on Communications (APCC)*, 2022, pp. 433-438.
- [3] N. Ilg, P. Duplys, and M. Menth, "A survey of contemporary open-source honeypots, frameworks, and tools," *Journal of Network and Computer Applications*, vol. 220, p. 103738, 2023.
- [4] R. Agrawal, S. Kopparapu, and S. G. V. R., "Long-Term Study of Honeypots in a Public Cloud," in *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2022, pp. 54-62.
- [5] P. Rajesh, P. Shrivastava, and B. S. Bindhumadhava, "Analysis Of Cyber Threat Detection And Emulation Using MITRE Attack Framework," in *International Conference on Intelligent Data Science Technologies and Applications*, 2022, pp. 112-125.

Appendix D: Script Source Code

enrichment_engine.py

```
import json
import requests
import pandas as pd
import time
import os
from datetime import datetime

# --- CONFIGURATION ---
# Replace with YOUR actual API Key from AbuseIPDB
API_KEY =
'b91c911091f53574a1c1f74353ef11c28f4c338ce7374b5fae2a603c2f0109e3c70ccf32029f056e'

# Path to T-Pot Cowrie logs (Standard T-Pot location)
LOG_FILE_PATH = '/home/kali/Desktop/data/cowrie/log/cowrie.json'
OUTPUT_FILE = 'enriched_threat_intel.csv'

# --- FUNCTIONS ---

def load_logs(filepath, max_lines=500):
    """Reads the last N lines of the log file to get recent attacks."""
    data = []
    print(f"[*] Reading logs from {filepath}...")

    if not os.path.exists(filepath):
        print(f"[!] Error: Log file not found at {filepath}")
        return []

    try:
        with open(filepath, 'r') as f:
            # Read line by line
            lines = f.readlines()
            # Take the last 'max_lines' to analyze recent activity
            for line in lines[-max_lines:]:
                try:
                    log_entry = json.loads(line)
                    # We only care about login attempts or command execution
                    if log_entry.get('eventid') == 'cowrie.login.failed':
                        data.append({
                            'timestamp': log_entry.get('timestamp'),
                            'src_ip': log_entry.get('src_ip'),
                            'username': log_entry.get('username'),
                            'password': log_entry.get('password')
                        })
                except json.JSONDecodeError:
                    continue
    except Exception as e:
        print(f"[!] Error reading file: {e}")

    print(f"[*] Found {len(data)} potential attack events.")
    return data

def check_ip_reputation(ip):
    """Queries AbuseIPDB for IP reputation."""
    url = 'https://api.abuseipdb.com/api/v2/check'
    querystring = {
        'ipAddress': ip,
```

```

    'maxAgeInDays': '90'
}
headers = {
    'Accept': 'application/json',
    'Key': API_KEY
}

try:
    response = requests.request(method='GET', url=url, headers=headers, params=querystring)
    if response.status_code == 200:
        decoded = response.json()
        return decoded['data']
    elif response.status_code == 429:
        print("[!] API Rate Limit Exceeded. Waiting...")
        time.sleep(60) # Wait if we hit a limit
        return None
    else:
        print(f"[!] API Error: {response.status_code}")
        return None
except Exception as e:
    print(f"[!] Connection Error: {e}")
    return None

# --- MAIN EXECUTION ---

def main():
    print("--- STARTING THREAT ENRICHMENT ENGINE ---")

    # 1. Load Raw Data
    raw_attacks = load_logs(LOG_FILE_PATH)
    if not raw_attacks:
        print("[!] No logs found. Exiting.")
        return

    # Convert to DataFrame for easier handling
    df = pd.DataFrame(raw_attacks)

    # Get unique IPs to avoid querying the same IP twice (saves API credits)
    unique_ips = df['src_ip'].unique()
    print(f"[*] Identified {len(unique_ips)} unique attacker IPs.")

    enriched_data = []

    # 2. Enrich Data (Loop through IPs)
    print("[*] Querying AbuseIPDB (This may take a moment)...")

    for i, ip in enumerate(unique_ips[:15]): # Limit to 15 IPs for testing
        print(f" [{i+1}/{min(len(unique_ips), 15)}] Checking IP: {ip}")

        reputation = check_ip_reputation(ip)

        if reputation:
            enriched_data.append({
                'IP': ip,
                'Country': reputation.get('countryCode'),
                'ISP': reputation.get('isp'),
                'Confidence_Score': reputation.get('abuseConfidenceScore'),
                'Total_Reports': reputation.get('totalReports'),
                'Last_Reported': reputation.get('lastReportedAt')
            })

```

```
# Sleep briefly to be nice to the API
time.sleep(1)

# 3. Save Results
if enriched_data:
    enrich_df = pd.DataFrame(enriched_data)
    enrich_df.to_csv(OUTPUT_FILE, index=False)
    print(f"\n[SUCCESS] Enriched Intelligence saved to: {OUTPUT_FILE}")
    print(enrich_df.head()) # Show preview
else:
    print("\n[!] No data was enriched.")

if __name__ == "__main__":
    main()
```

mitre_engine.py

```
import json
import requests
import pandas as pd
import time
import os

# --- CONFIGURATION ---
API_KEY =
'b91c911091f53574a1c1f74353ef11c28f4c338ce7374b5fae2a603c2f0109e3c70ccf32029f056e' # <---
PASTE YOUR KEY HERE
OUTPUT_FILE = 'unified_threat_feed.csv'

# Log Paths for T-Pot Honeypots
LOG_SOURCES = {
    "Cowrie": "/home/kali/Desktop/data/cowrie/log/cowrie.json",
    "Dionaea": "/home/kali/Desktop/data/dionaea/log/dionaea.json",
    "Suricata": "/home/kali/Desktop/data/suricata/log/eve.json"
}

# --- UNIFIED MITRE ATT&CK MAPPING ---
# Expanded dictionary to cover SSH, SMB, HTTP, and Network Exploits
MITRE_MAPPING = {
    # Cowrie (SSH/Telnet)
    "cowrie.login.failed": {"ID": "T1110", "Technique": "Brute Force"},
    "cowrie.session.connect": {"ID": "T1046", "Technique": "Network Service Discovery"},
    "wget": {"ID": "T1105", "Technique": "Ingress Tool Transfer"},
    "curl": {"ID": "T1105", "Technique": "Ingress Tool Transfer"},

    # Dionaea (SMB/FTP/HTTP)
    "smbd": {"ID": "T1210", "Technique": "Exploitation of Remote Services"},
    "httpd": {"ID": "T1190", "Technique": "Exploit Public-Facing Application"},
    "ftpd": {"ID": "T1021", "Technique": "Remote Services"},
    "mqtt": {"ID": "T1046", "Technique": "Network Service Discovery"},

    # Suricata (Network Alerts)
    "alert": {"ID": "T1595", "Technique": "Active Scanning"},
    "Exploit": {"ID": "T1210", "Technique": "Exploitation of Remote Services"},
    "Malware": {"ID": "T1204", "Technique": "User Execution"}
}

# --- FUNCTIONS ---

def get_mitre_tag(event_type, detailed_info=""):
    """Maps varied event types to MITRE Tags."""
    # 1. Check detailed info (Command or Alert Signature)
    if detailed_info:
        for keyword, mapping in MITRE_MAPPING.items():
            if keyword.lower() in str(detailed_info).lower():
                return mapping['ID'], mapping['Technique']

    # 2. Check Event Type
    for keyword, mapping in MITRE_MAPPING.items():
        if keyword in event_type:
            return mapping['ID'], mapping['Technique']

    return "T1071", "Standard Application Layer Protocol" # Fallback

def parse_line(line, honeypot_name):
    """Normalizes logs from different honeypots into a standard format."""
```

```

try:
    log = json.loads(line)
    data = {
        'timestamp': log.get('timestamp', 'N/A'),
        'src_ip': None,
        'event': 'unknown',
        'details': None
    }

    # --- COWRIE PARSING ---
    if honeypot_name == "Cowrie":
        data['src_ip'] = log.get('src_ip')
        data['event'] = log.get('eventid')
        data['details'] = log.get('input') # Commands

    # --- DIONAEA PARSING ---
    elif honeypot_name == "Dionaea":
        # Dionaea often uses 'remote_host' or 'src_ip'
        data['src_ip'] = log.get('src_ip') or log.get('remote_host')
        data['event'] = log.get('connection_type') or log.get('protocol')
        data['details'] = log.get('connection_transport')

    # --- SURICATA PARSING ---
    elif honeypot_name == "Suricata":
        data['src_ip'] = log.get('src_ip')
        data['event'] = log.get('event_type')
        if data['event'] == 'alert':
            # Extract the specific alert signature
            data['details'] = log.get('alert', {}).get('signature')

    return data
except:
    return None

def load_all_logs(max_lines=300):
    """Iterates through all configured log sources."""
    unified_data = []

    for name, path in LOG_SOURCES.items():
        print(f"[*] Reading {name} logs from {path}...")

        if not os.path.exists(path):
            print(f"    [!] File not found. Skipping {name}.")
            continue

        try:
            with open(path, 'r') as f:
                lines = f.readlines()[-max_lines:] # Read last N lines
                for line in lines:
                    parsed = parse_line(line, name)

                    if parsed and parsed['src_ip']:
                        # Filter out internal IPs/Noise
                        if parsed['src_ip'] == "127.0.0.1": continue

                        # Apply MITRE Mapping
                        t_id, tech = get_mitre_tag(str(parsed['event']), parsed['details'])

                        unified_data.append({
                            'Timestamp': parsed['timestamp'],

```

```

        'Honeygot': name,
        'Src_IP': parsed['src_ip'],
        'Event': parsed['event'],
        'Details': parsed['details'],
        'MITRE_ID': t_id,
        'MITRE_Technique': tech
    })
except Exception as e:
    print(f"  [!] Error reading {name}: {e}")

print(f"[*] Total events captured across all honeypots: {len(unified_data)}")
return unified_data

def check_ip_reputation(ip):
    url = 'https://api.abuseipdb.com/api/v2/check'
    headers = {'Accept': 'application/json', 'Key': API_KEY}
    params = {'ipAddress': ip, 'maxAgeInDays': '90'}
    try:
        r = requests.get(url, headers=headers, params=params)
        if r.status_code == 200: return r.json()['data']
        elif r.status_code == 429: time.sleep(5); return None
    except: return None

# --- MAIN EXECUTION ---
def main():
    print("---- UNIFIED THREAT INTELLIGENCE ENGINE (v2.0) ----")

    # 1. Load & Normalize Data
    events = load_all_logs()
    if not events: return

    # 2. Enrich Data (Limit API calls for demo)
    df = pd.DataFrame(events)
    unique_ips = df['Src_IP'].unique()[:10] # Process top 10 unique IPs

    ip_intel = {}
    print(f"[*] Enriching {len(unique_ips)} unique IPs via AbuseIPDB...")

    for ip in unique_ips:
        print(f"  checking: {ip}")
        intel = check_ip_reputation(ip)
        if intel:
            ip_intel[ip] = intel
            time.sleep(1)

    # 3. Merge Intelligence
    final_results = []
    for event in events:
        if event['Src_IP'] in ip_intel:
            intel = ip_intel[event['Src_IP']]
            event['Country'] = intel.get('countryCode')
            event['Confidence'] = intel.get('abuseConfidenceScore')
            final_results.append(event)

    # 4. Save
    if final_results:
        final_df = pd.DataFrame(final_results)
        # Reorder columns for readability
        cols = ['Timestamp', 'Honeygot', 'Src_IP', 'Country', 'Confidence', 'Event', 'Details', 'MITRE_ID',
        'MITRE_Technique']

```



```
    final_df = final_df[cols]
    final_df.to_csv(OUTPUT_FILE, index=False)
    print(f"\n[SUCCESS] Master Feed generated: {OUTPUT_FILE}")
    print(final_df.head())
else:
    print("[!] No enriched data to save.")

if __name__ == "__main__":
    main()
```

misp_connector.py

```
import pandas as pd
import json
import uuid
from datetime import datetime

# --- CONFIGURATION ---
INPUT_FILE = 'unified_threat_feed.csv'
OUTPUT_FILE = 'misp_import_data.json'

# --- FUNCTIONS ---

def generate_misp_event(row):
    """Converts a single CSV row into a MISP Event structure."""

    # 1. Create unique Event ID
    event_uuid = str(uuid.uuid4())

    # 2. Map Honeypot Data to MISP Attributes
    attributes = []

    # Attribute: Source IP
    attributes.append({
        "type": "ip-src",
        "value": row['Src_IP'],
        "comment": f"Detected by {row['Honeypot']}",
        "to_ids": True # This means "Yes, this is an Indicator of Compromise"
    })

    # Attribute: MITRE Technique (Tagging)
    if row.get('MITRE_ID'):
        attributes.append({
            "type": "text",
            "value": f"MITRE ATT&CK: {row['MITRE_ID']} - {row['MITRE_Technique']}",
            "comment": "Automated Classification"
        })

    # Attribute: Country of Origin
    if row.get('Country'):
        attributes.append({
            "type": "text",
            "value": f"Origin: {row['Country']}",
            "comment": "GeoIP Enrichment"
        })

    # 3. Construct the Full Event Object
    misp_event = {
        "Event": {
            "uuid": event_uuid,
            "info": f"T-Pot Alert: {row['Event']} detected from {row['Src_IP']}",
            "date": datetime.now().strftime("%Y-%m-%d"),
            "threat_level_id": "2", # Medium
            "analysis": "1", # Initial Analysis
            "Attribute": attributes
        }
    }

    return misp_event

# --- MAIN EXECUTION ---
```

```

def main():
    print("--- STARTING MISP CONNECTOR (INTEGRATION MODULE) ---")
    print(f"[*] Loading intelligence from {INPUT_FILE}...")

    try:
        df = pd.read_csv(INPUT_FILE)
        print(f"[*] Loaded {len(df)} enriched threat events.")
    except FileNotFoundError:
        print("[!] Error: CSV file not found. Run the enrichment engine first.")
        return

    misp_export_list = []

    print("[*] Transforming data to MISP JSON format...")

    # Process the first 10 rows as a sample for the report
    for index, row in df.head(10).iterrows():
        event = generate_misp_event(row)
        misp_export_list.append(event)

    # Save the output
    with open(OUTPUT_FILE, 'w') as f:
        json.dump(misp_export_list, f, indent=4)

    print(f"[*] Transformation Complete.")
    print(f"[SUCCESS] {len(misp_export_list)} events formatted for MISP ingestion.")
    print(f"[SUCCESS] Output saved to: {OUTPUT_FILE}")

    # Preview for the user
    print("\n--- PREVIEW OF MISP EVENT (Row 1) ---")
    print(json.dumps(misp_export_list[0], indent=4))

if __name__ == "__main__":
    main()

```

misp_push.py

```
import json
from pymisp import PyMISP, MISPEvent, MISPObject
import urllib3

# Suppress SSL warnings for local servers
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

# --- CONFIGURATION ---
MISP_URL = 'https://localhost'
MISP_KEY = 'API_KEY'
INPUT_FILE = 'misp_import_data.json'
VERIFY_SSL = False

# --- SIMULATION MODE ---
# Set to True to demonstrate functionality without a live server
SIMULATION_MODE = False

def push_to_misp():
    print("--- STARTING PYMISP INJECTION MODULE ---")

    # 1. Initialize PyMISP (The Connector)
    print(f"[*] Initializing connection to MISP Server ({MISP_URL})...")
    if SIMULATION_MODE:
        print(" [NOTE] Simulation Mode Active: Skipping network handshake.")
    else:
        try:
            misp = PyMISP(MISP_URL, MISP_KEY, ssl=VERIFY_SSL)
        except Exception as e:
            print(f"[!] Connection Failed: {e}")
            return

    # 2. Load the JSON Data
    try:
        with open(INPUT_FILE, 'r') as f:
            events_data = json.load(f)
        print(f"[*] Loaded {len(events_data)} events from {INPUT_FILE}.")
    except FileNotFoundError:
        print("[!] Error: JSON file not found.")
        return

    # 3. Iterate and Push
    print("[*] Processing events queue...")

    success_count = 0
    for i, entry in enumerate(events_data):
        # Extract the inner event data
        raw_event = entry.get('Event', {})

        # Create a real PyMISP Event Object
        misp_event = MISPEvent()
        misp_event.info = raw_event.get('info')
        misp_event.threat_level_id = raw_event.get('threat_level_id')
        misp_event.analysis = raw_event.get('analysis')

        # Add Attributes
        attr_count = 0
        for attr in raw_event.get('Attribute', []):
            misp_event.add_attribute(
                type=attr['type'],
```

```

        value=attr['value'],
        comment=attr.get('comment'),
        to_ids=attr.get('to_ids', False)
    )
    attr_count += 1

# Simulate the Push
print(f"    [>] Pushing Event {i+1}: '{misp_event.info}' ({attr_count} attributes)")

if SIMULATION_MODE:
    # Fake a successful server response
    print(f"    [SUCCESS] Event ID: {1000+i} created on MISP.")
    success_count += 1
else:
    # This code would run on a real server
    response = misp.add_event(misp_event)
    if 'Event' in response:
        print(f"    [SUCCESS] Event created.")
        success_count += 1

print("\n--- INTEGRATION SUMMARY ---")
print(f"[SUCCESS] {success_count} Threat Intelligence packages successfully processed.")
print("[*] MISP Integration Module: ONLINE")

if __name__ == "__main__":
    push_to_misp()

```